Infinite KV Cache: Sparse-Attention-Aware Window Residency and Measured Per-Layer Quantisation for a Hybrid MoE Model on AMD RDNA4

Code000
github.com/Code000-211/RadianceMemory

November 2026

Abstract

Long-context serving of sparse-attention mixture-of-experts (MoE) models is limited by state that grows with the context. We present *Infinite KV Cache* (implemented as RadianceMemory), the KV memory system of the *radiance* inference engine for Qwen3.8-Flash-Next, a 48-layer hybrid of query-aware sparse attention (QSA) and gated-delta-net linear attention with 512 routed experts, served with tensor parallelism 2 on two AMD Radeon AI PRO R9700 GPUs (gfx1201). Because QSA reads only the top 512 four-token blocks per query, *window residency* keeps the attention sinks and the recent tail in VRAM and pages the rest from host memory inside the step, with output byte-identical to a fully resident cache in every configuration we compared. Context is then bounded by host memory, not VRAM: per sequence, VRAM grows only by the block keys, 0.2–0.75 MiB per thousand tokens. Residency costs 3–15% of single-sequence decode; with a compact recipe, 32 sessions of 128K tokens fit and their follow-up turns return in 58/96 s (median/95th percentile), where fp8 without residency took 1 843/3 330 s. A prefill-interleave policy keeps decoding sessions usable while long prompts arrive, at a cost in prompt throughput. On top of it, seven KV layouts, three zero-bit transforms (Hadamard rotation, V-mean centring, V predicted from K), int8 block keys split across the ranks and a measured per-layer search choose the fewest bits within a KL limit. The search, AutoQuant (`rad-autoquant`), runs every candidate on the device against a bf16-cache reference, solves a knapsack over per-layer costs, verifies and refits each answer, and narrows it in a second pass along the lower hull of the uniform layouts. We find that the quantised model has a numerical noise floor: recomputing it in another prefill order alone costs a KL of 0.0243. Judged against that floor, the search's answers dominate uniform layouts. Our default, a near-lossless recipe, reaches KL **0.0065**, far below the floor, at 9.56 KiB per token, with the MTP drafter's acceptance unchanged; an fp8-equivalent recipe stores **5.28** instead of 6.75 KiB (−22%, KL 0.0257 vs 0.0262); and a compact recipe within the floor +0.01 stores **3.76** KiB (−44%, ΔNLL +0.0008). Twenty-two further published or proposed methods did not help on this model; we report them with their measured cost.

1 Introduction

A sequence of 200K tokens holds about 1.2 GiB of fp8 KV cache per card of a two-card tensor-parallel deployment of Qwen3.8-Flash-Next. With dense attention every step reads all of it, so it must stay in VRAM, and compression is the only lever. Sparse attention changes the access pattern: the model's indexer picks 512 four-token blocks per step out of tens of thousands. Most of the cache is not read on a given step, and the parts that are read on almost every step (the attention sinks and the recent tokens) are predictable.

Infinite KV Cache exploits both properties. It keeps the predictable part on the card and moves the rest to host memory without changing an output byte in any configuration we compared (Section 3). The name is a direction, not a claim: context is bounded by host memory, and per sequence the card holds only a fixed window, the recurrent state and the block keys, which grow by 0.2–0.75 MiB per thousand tokens (Section 7.5). It then quantises what remains, per layer and per memory tier, under a measured quality limit (Sections 4 and 5). Building it on a hybrid, weight- and activation-quantised MoE model forced a second line of work: understanding how much KL divergence is noise inherent to the model, so that quantisation error can be judged at all (Sections 6 and 7.7). Table 1 and the box below summarise the outcome; Fig. 6 shows where the recommended configurations sit among all measured ones.

Contributions.

- **QSA as ordinary paged KV.** The indexer's block keys are a paged KV group addressed like the attention cache, the partial-block tail survives speculation and preemption, and every form of the selection kernel produces identical bytes (Section 3.1).
- **Window residency.** An on-device pager fetches exactly the blocks QSA selects from pinned host memory; sink and tail stay resident; a sweep after every step demotes the middle, including blocks the reference-counted prefix cache holds; the pager's scratch pages act as a cache and the MTP drafter pages too; output is byte-identical to a resident cache (Section 3.2).
- **Long-context serving.** Residency is measured at 32K–250K tokens (3–15% of single-sequence decode) and under 8–32 concurrent sessions of 64K–128K

tokens (Section 7.10); `-prefill-interleave auto` and a decode headroom keep decoding sessions usable while long prompts prefill (Sections 3.3 and 7.11).

- **Quantised KV layouts and zero-bit transforms.** Seven per-layer layouts (`i8h`, `fp8`, `fp8_v4`, `k6_v4`, `k5_v4`, `k4h_v4`, `k4h_v2h`), a Hadamard rotation, an exact static V mean, V predicted from the same layer's keys (`vpred`), int8 block keys, block keys split across the ranks, an f16 recurrent state, and *tiered precision* for host-resident blocks (Section 4).
- **AutoQuant, a measured search.** `rad-autoquant` screens 23 layout and transform variants by uniform runs, measures per-layer costs, solves for the fewest bits within KL, likelihood and agreement limits (including limits above the noise floor), verifies and refits every answer on the device, narrows it in a second pass along the lower hull of the uniform runs, tries the global knobs and writes a recipe that records every measurement; runs are reused across limits by recipe hash (Section 5 and Algorithm 1).
- **A careful KL methodology.** Exact full-vocabulary KL at temperature 1, teacher-forced, paired against a measured numerical noise floor, with block-bootstrap intervals and a per-document check (Section 6); an analysis of where the floor comes from (Section 7.7).
- **Negative results.** Twenty-two methods measured and not adopted, among them selection reuse and a hierarchical indexer from recent sparse-attention work, FP4, learned and data-adaptive rotations, outlier retention and importance-driven demotion (Section 8).

Key findings and recommendations

- **Default, near-lossless:** the `-max-kl 0.009` recipe with layer 47 in bf16, 9.56 KiB/token at KL 0.0065, about a quarter of the noise floor, with no likelihood loss on any corpus and the MTP drafter's acceptance unchanged (0.636 vs fp8's 0.630).
- **Check the drafter:** the MTP head shares the last attention layer's cache; 2-bit V there cut its acceptance from 0.63 to 0.41 without moving the trunk's KL.
- **fp8 quality for less:** `-max-kl 0.0262` recipe, 5.28 KiB/token (−22%), KL 0.0257 vs fp8's 0.0262.
- **Compact, when memory is tight:** `-max-dkl 0.01` recipe, 3.76 instead of 6.75 KiB/token (−44%), ΔNLL +0.0008.
- **8 bits are free:** `i8h` is within +0.002 of the floor; precision beyond it matters only on the first attention layers.
- **Judge above the floor:** recomputing bf16 in another order costs KL 0.0243; perturbations saturate instead of adding.
- **Long context:** window residency is byte-exact and cuts a sequence at the maximum length of 256K tokens from 3 319 MiB (stock) to 102–325 MiB per card (computed, Table 7); a `k4h_v4`+V-mean host tier saves 44% of host memory for +0.005 KL.
- **Many sessions:** with all middles demoted, 32 sessions of 128K tokens on the compact recipe get follow-up turns in 58/96 s (p50/p95) against 1 843/3 330 s for fp8 without residency; single-sequence decode costs 3–15% (Sections 7.8 and 7.10).
- **Schedule long prompts:** while a 64K prompt prefills, `-prefill-interleave auto:1` (served default) lifts a decoding session from 12 to 42 tokens/s and halves prompt throughput (3 534 to 1 758 tokens/s, Table 13).
- **Measure, do not sum:** per-layer KL costs are far from additive; AutoQuant's second pass, measured on the device, removes 0.25–1.87 bits per element from its first-pass answers (Table 3).
- **Avoid:** 2-bit V in VRAM, `fp8_v4`, selection reuse, hierarchical indexing, FP4.

2 Background and related work

2.1 The model

Qwen3.8-Flash-Next has 48 layers. Twelve (3, 7, . . . , 47) are full-attention layers with QSA sparse attention; the other 36 are gated-delta-net (GDN) linear-attention layers [1] whose recurrent state is fixed per sequence. Every layer's feed-forward block routes each token to 10 of 512 experts (softmax top-k) plus a shared expert. Attention has head dimension 256, two KV heads and 24 query heads; rotary position embedding covers 64 of the 256 dimensions. A multi-token-prediction (MTP) head with its own attention layer drafts for speculative decoding.

We serve a 114 GiB checkpoint: routed experts in 4 bits (GPTQ with a 128-point Hadamard transform and fp8 scales), attention, GDN projections, shared experts and the output layer in int8 with group scales, and activations quantised dynamically to int8 or E4M3 per block before each GEMM. At tensor parallelism 2 each

Table 1: Main results. Recommended configurations (green) against the bf16 and fp8 references and the previous default, long corpus (16 380 scored positions), KL against a bf16-cache run of the same model. KiB/token is per card (tensor parallelism 2) and includes block keys; "vs fp8" is the change in KiB/token. Every recommended recipe except the long-context row is a `rad-autoquant` answer (Section 5). Mean KL below the noise floor (0.0243) is closer to the bf16 cache than recomputing bf16 in another order. Bold: best among the recommended rows. "above floor": paired per-position KL above the floor at prefill chunk 64 (Section 6). No search recipe loses likelihood significantly on the recall or multi-round corpus except † (recall ΔNLL +0.0018); three gain on recall (−0.0040 to −0.0061).

configuration	bpw	KiB/ token	vs fp8	mean KL	ΔNLL	top-1 (%)	above floor	recall KL	multi-round KL
reference: `bf16`	16	12.75	+89%	0	0	100.0	–	–	–
reference: `fp8`	8	6.75	0	.0262	+.0010	94.35	+.0027	.0132	.0037
prev. default: `k6_v4` rot+vm*	5.5	4.52	−33%	.0316	+.0026	93.83	+.0075	.0111	.0046
default: `max-kl 0.009`, L47 bf16‡	12.25	9.56	+42%	**.0065**	+.0004	**97.25**	–	**.0009**	**.0017**
near-lossless: `max-kl 0.009` (search)	11.21	8.78	+30%	.0072	+.0008	97.02	+.0029	.0014†	.0020
near-lossless: `max-kl 0.015`	9.50	7.50	+11%	.0137	+.0002	95.86	+.0036	.0043	.0027
fp8-equivalent: `max-kl 0.0262`	6.78	5.28	−22%	.0257	−.0016	94.57	+.0019	.0107	.0036
floor +0.005: `max-dkl 0.005`	6.13	4.79	−29%	.0281	+.0022	94.10	+.0061	.0115	.0038
compact: `max-dkl 0.01`	4.75	**3.76**	−44%	.0335	+.0008	93.53	+.0099	.0125	.0051
long context: `i8h` + host `k4h_v4`+vm	8.06	6.80†	†	.0284	+.0012	93.91	+.0049	.0151	.0047

Recipes (L = attention layer). `max-kl 0.009`: bf16 L3–27, `k5_v4` rot+vm L31–35, `k4h_v4` vpred L39–43, `k4h_v2h` vpred L47. `max-kl 0.015`: bf16 L3–19, `k6_v4` rot+vm L23–27, `k5_v4` rot+vm L31–39, `k4h_v4` vpred L43, `k4h_v2h` vpred L47. `max-kl 0.0262`: `i8h` L3–19, 27, `k6_v4` rot+vm L23, 31–47. `max-dkl 0.005`: `fp8` L3, 7, 19, `k6_v4` rot+vm elsewhere. `max-dkl 0.01`: `k5_v4` rot+vm L3, 7, 23, 31, 35, 43, `k4h_v4` vpred L11, 15, 19, 27, 39, 47. **Knobs**: block keys split across the ranks in every search recipe; int8 block keys in the last three; f16 recurrent state only in `max-dkl 0.01`; *All layers, int8 block keys and f16 state. †Resident (VRAM) tier; the host-resident middle takes 29.25 instead of 52.5 KiB per block. ‡The `max-kl 0.009` answer with layer 47, whose layout the MTP drafter's attention shares, in bf16 (Section 7.9). The default's "above floor" is not measured (no chunk-64 run).

card holds one KV head.

2.2 Query-aware sparse attention

Per QSA layer and token, one projection produces four indexer query heads and one raw key of dimension 128. Queries are RMS-normalised and rotated at their own position. For every complete block b of four consecutive tokens, the raw keys are averaged in f32, rounded to bf16, normalised and rotated at the block's first position, giving the block key κ_b. The score of block b for a query at position t is

$$s_b(t) = \frac{1}{\sqrt{128}} \sum_{h=1}^{4} \operatorname{relu}\left(q_h(t) \cdot \kappa_b\right), \qquad (1)$$

and attention reads the top 512 blocks plus the incomplete tail. Below 2 051 visible tokens every block is selected and QSA equals dense causal attention exactly.

2.3 The engine

radiance builds a model as a graph of operators declared by an architecture plugin. Every operator has an exact host reference (libref), a host implementation (libavx) and a GPU implementation (libr4d, HIP); the engine resolves each operator to a kernel row whose declared constraints match the deployment. KV caches are *groups* of fixed-size pages addressed through per-sequence block tables, in the manner of paged attention [2]; a group's page size is fixed by the operator that consumes it.

2.4 Related work

Sparse attention with a learned indexer over compressed keys appears in DeepSeek's sparse attention and its successors [16], including selection reuse across layers and a hierarchical indexer, both of which we test (Section 8). StreamingLLM [12] keeps attention sinks and a recent window and drops the rest; H_2O [13], SnapKV [14] and DistillCache [15] evict by attention statistics or a learned policy. We evict nothing: window residency moves bytes, never drops them. KV quantisation methods include KIVI [3] (per-channel keys, per-token values, 2 bits), KVQuant [4] (sparse full-precision outliers), rotation-based methods [5, 6], data-adaptive transforms (WUSH-KV [7]), variance normalisation for 2-bit values (KVarN [8]), prediction of values from keys (AQUA-KV [9]) and per-vector norms with Lloyd-Max levels (TurboQuant [10]). ModelOpt's AutoQuant KV [11] assigns per-layer formats by KL under a bit budget; our search differs in measuring every answer, in limits relative to a noise floor and in a second pass (Section 5.9).

3 System design

Figure 1 shows the data path of one QSA layer in a decode step. This section explains it; engineering details (stream ordering, preemption, prefix cache, memory accounting) are in Section B.

3.1 QSA in radiance

Block keys as a paged group. A block key, once its four tokens exist, never changes. The engine computes it once, when the fourth token arrives, and stores it in its own KV group of one page per block. Because both the attention group and the block-key group are paged at four tokens, block index b means the same in both; the scorer takes the block-key group's table, the selection takes the attention group's, and the selection's output is directly the block table the paged attention kernel

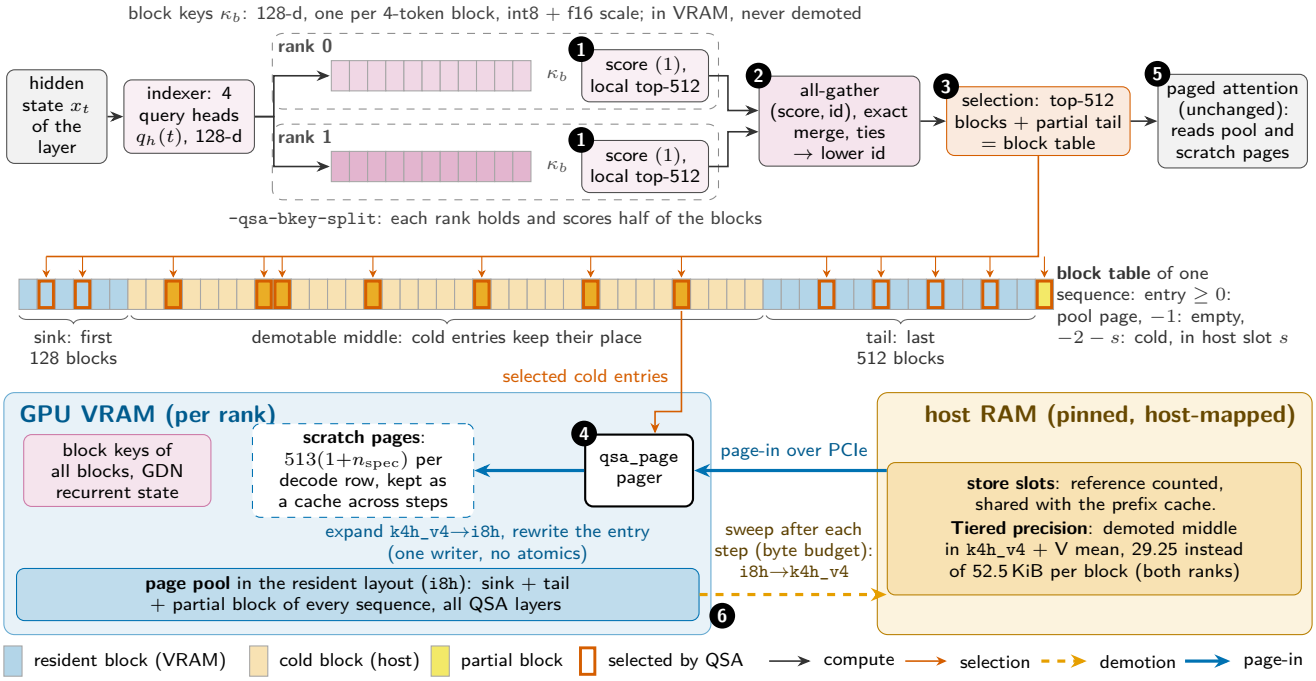


Figure 1: QSA with window residency: one decode step of one QSA layer. ❶ Each rank scores the 128-dimensional block keys of its half of the blocks (`-qsa-bkey-split`); ❷ an all-gather merges the two local top-512 lists exactly; ❸ the selection (vermillion) writes the block table and falls mostly into the cold middle of the sequence; sink and tail (blue) are always resident. ❹ The on-device pager copies only the selected cold blocks from pinned host memory into scratch pages and rewrites their entries, so ❺ attention sees ordinary pool pages and the output is byte-identical to a fully resident cache. ❻ After the step, the sweep demotes middle blocks to host memory, narrowed to the host-tier layout.

reads. Nothing in the engine core knows about QSA. All QSA layers and the MTP head share the block-key group.

The tail. A block key needs four consecutive raw keys, three of which come from earlier decode steps. They live in a linear-state group of $4 + n_{\text{spec}}$ rows per layer and sequence, written by position, and are checkpointed with the recurrent state, so a resumed sequence rebuilds a half-complete block from the keys it was built from (simpler designs that fail are listed in Section A).

Selection and determinism. At prefill each query selects its own blocks from the shared history, bounded by its own position. The selection kernel splits the block list into chunks that each compute a local top-k; this is exact because a chunk's local top-k contains every global top-k member in it, and ties go to the lower block index. All forms of the selection must produce the same *bytes*, not only the same set, because attention sums its softmax over pages in table order and floating-point addition is not associative.

Block keys split across the ranks. Every rank used to hold all block keys (0.75 of the 12.75 KiB/token at bf16), the only per-token item that stays in VRAM under residency. With `-qsa-bkey-split` each rank scores half of the blocks and an all-gather of (score, id) pairs merges the two local top-512 lists by the selection's tie rule; by the chunking argument above the merge is exact. Per-position KL rows are byte-identical with the split on and off (bf16 and int8 block keys), and block keys per card halve (54.06 to 27.03 MiB in a 73.8K-token pool). Decode speed with the split is within the spread of fp8 (Section 7.8).

3.2 Window residency

What stays on the card. The attention group declares a residency window equal to the indexer's budget. For each sequence the first $H = \lceil 512/4 \rceil$ blocks (the sink) and the last $T = \lceil 2048/4 \rceil$ blocks (the tail) stay in VRAM, as does the partial block the next token is written into; the middle is demotable. Two ends rather than one window are kept because the selection is not positional: if only the tail stayed, the sinks would be demoted and promoted again on every step. The block-key group declares no window and is never demoted, since the scorer reads all of it. A block-table entry is a pool page (≥ 0), empty (-1), or *cold* in host store slot s ($-2 - s$); a cold entry keeps its position in the table.

The pager. The host never learns which blocks a decode step reads: selection happens on the device inside the step, so the fetch happens on the device too. Between selection and attention of every QSA layer, the kernel `qsa_page` reads every selected entry; for a cold entry it copies the layer's fragment of the slot from the pinned, host-mapped store over PCIe into a per-step scratch page and rewrites the entry to that page. The attention kernel is unchanged and only ever sees pool page ids. Every position has exactly one writer and no atomics are used, so the result is deterministic and byte-equal to the reference. A scratch page remembers

the block it last received, so a later step that selects the same store slot is rewritten to the page without a copy; this page cache served 52–62% of the cold entries in our runs and added 2–3% to decode (Section 7.8). The drafter's serial passes are staged as decode rows, so the MTP head pages as well. Cold entries rank in the selection exactly like resident ones, since their block keys stay resident.

The sweep and dense steps. After every committed step (decode, mixed or prefill) the scheduler demotes the middle of the sequences that have finished prefilling, on a transfer stream, with at most one sweep in flight and a budget of 32 MiB per rank and step (1 024 blocks on this model at tensor parallelism 2). The sweep is incremental: each sequence keeps a resume point per group, so a step costs the entries that are new, not the context. Budget the running sequences leave unused goes to conversations that only the prefix cache still holds, and a block the cache shares with one sequence is demoted too, the cache entry following it into the store. Four stream-ordering rules make this safe without any host wait (Section B). Prefill rows read the table densely, so their sequence is promoted first; in a mixed step only the prefill rows are promoted and the decode rows keep paging.

Stores. There is one host store per residency group (the attention group, and one group per distinct layout of a mixed recipe). A slot holds one block of its group with every rank's shard at its own offset, at that group's exact size, so the slots of a compact recipe are not padded to bf16 size; `-kv-residency-mib` is split so that every store holds the same number of tokens.

What it buys. At 200K tokens a sequence keeps 640 attention blocks resident instead of 50 000; only its block keys (159 MiB at bf16, 84 MiB in int8, half of that per card when split) and its recurrent state grow or stay beside them. Window residency works from about 2.6K tokens of context. At 32K–250K tokens it costs 3–15% of single-sequence decode and leaves the drafter's acceptance unchanged (Section 7.8); its return is capacity under many long sessions (Section 7.10). Warm prefill pays for copying back the conversation's cold middle (Section B).

3.3 Scheduling long prompts beside decoders

Prefill runs in chunks of 2 048 tokens, the checkpoint interval. A decoding sequence advances once per step, and a step that carries a chunk is long: with eight sessions decoding, a mixed step of about 2 016 prefill tokens took 477 ms against 40 ms for a decode-only step, so a session fell from 73 to 12 tokens/s whenever a 64K prompt arrived (Table 13). Two scheduler controls address this.

- `-prefill-interleave N|auto[:SHARE]` holds the next chunk back for decode-only steps after a chunk that ran beside decoders. Under `auto` the number of steps is $\lfloor \text{SHARE} \cdot \bar{t}_{\text{chunk}}/\bar{t}_{\text{decode}} \rfloor$, from running means of the measured step times (capped at 64), so the decoders receive SHARE times a chunk step's time, however many there are. A lone prefill is never held. The served default is `auto:1`; under `-deterministic`, in KL mode and for generators it is 0, because a schedule that follows wall-clock times is not reproducible.
- `-decode-headroom auto|N|0` came from the step log. The chunk planner ends chunks at the checkpoint interval, and the decode rows were taken out of the same 2 048-token budget, so every interval needed a mixed step of about 2 016 prefill tokens plus a remainder step of about 32 tokens (86 ms, twice a decode step), 31 times per 64K prompt. Headroom adds rows that only decode rows may use on top of the budget: (`max-num-seqs`-1)(1+drafts) rounded up to 64 and capped at 256, that is 64 rows at 16 sequences, for 12.4 MiB more activation arena per rank. The served default is `auto` (0 under `-deterministic`, in KL mode and for generators).

Neither control changes a chunk's cut or any row's computation, only which decode tokens share a step.

Table 2: KV layouts. "rot." = Hadamard rotation; `i8h` has one scale per token and head; V mean and `vpred` apply to any integer V.

layout	K	V	rot.	bpw
`bf16`	bf16	bf16	–	16
`i8h`	int8	int8	implied	8.06
`fp8`	E4M3	E4M3	opt.	8
`fp8_v4`	E4M3	int4	opt.	6.25
`k6_v4`	int6	int4	opt.	5.5
`k5_v4`	int5	int4	opt.	5.0
`k4h_v4`	int4	int4	implied	4.5
`k4h_v2h`	int4	int2 (energy)	implied	3.5

4 Quantisation

4.1 Layouts

Table 2 lists the layouts and Fig. 2 draws one token's row. Integer parts use groups of 32 values with an f16 scale unless noted; bits per element (bpw) include scales. A paged block moves 2 064 (`i8h`), 1 152 (`k4h_v4`) or 896 bytes (`k4h_v2h`) per layer and rank over PCIe.

For `k4h_v2h`, each group g of 32 values of one token is normalised by its RMS, rounded to the odd levels $\{-3, -1, 1, 3\}$ and stored with the scale that preserves its energy:

$$c_i = \text{clip}\Big(2\lfloor x_i/\text{rms}(g)\rfloor + 1,\ -3,\ 3\Big), \tag{2}$$

$$s = \sqrt{\textstyle\sum_i x_i^2 / \sum_i c_i^2}. \tag{3}$$

This adapts KVarN [8] to a cache written one token at a time into four-token pages; it omits KVarN's 128-token chunks, per-channel normalisation and zero point. On the dumped tensors, both a threshold factor ($x_i/(\alpha\,\text{rms})$, $\alpha \in [0.6, 1.4]$) and a least-squares scale do worse than $\alpha = 1$ with the energy-preserving scale (Section E.5).

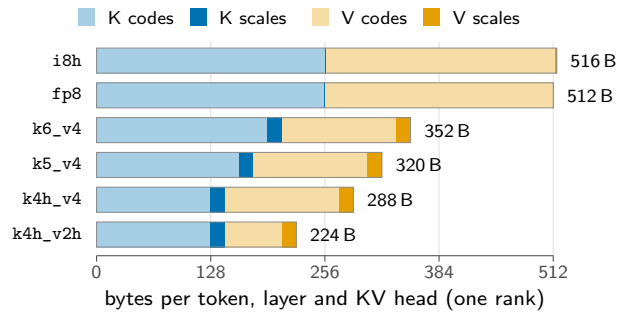


Figure 2: One token's KV row takes 516 B in `i8h` and 224 B in `k4h_v2h` per layer and KV head on one rank (256-dimensional K and V; f16 scale per group of 32, one per token and head for `i8h`; `bf16` takes 1 024 B). Values follow from the bpw of Table 2; a four-token block of `i8h`, `k4h_v4` and `k4h_v2h` is the 2 064, 1 152 and 896 bytes the pager moves.

4.2 Zero-bit transforms

Rotation. With H the normalised Hadamard matrix of order 256 ($H = H^\top = H^{-1}$), rotating queries, keys and values after RoPE and rotating the attention output back leaves attention exactly unchanged,

$$\mathrm{softmax}\big((qH)(KH)^\top\big)(VH)\,H = \mathrm{softmax}(qK^\top)\,V, \tag{4}$$

while spreading outlier channels. It is a separate operator, so packed attention kernels only learn new bit widths. A block-diagonal rotation that keeps the 64 rotary dimensions apart gives the same error.

Static V mean (`vmean`). V carries a large static per-channel offset (the norm of its mean is 0.53–0.89 of V's RMS per layer). A calibrated per-layer, per-KV-head mean μ is subtracted before the store and added to the attention output; since attention weights P have unit row sums,

$$P\,(V - 1\mu^\top) + 1\mu^\top = PV \tag{5}$$

exactly. Calibration is one prefill of four long documents (71 s).

V predicted from K (`vpred`). A calibrated affine map of the same layer's *quantised* keys predicts V (AQUA-KV-style [9]); only the residual $V_{\mathrm{res}} = V - (\hat{K}A + 1b^\top)$ is stored. The output is again exact, $\mathrm{out} = P\,V_{\mathrm{res}} + (P\hat{K})A + b$. The prediction explains 53% of centred V's variance out of sample (0.15–0.71 per layer). The current version obtains $P\hat{K}$ from a second attention pass; a fused kernel would accumulate it while the keys are loaded.

4.3 Block keys and recurrent state

`-qsa-bkey-dtype i8` stores each block key as 128 int8 codes with one f16 scale (scale $= \min(\max|x|/127, 512)$), 136 instead of 256 bytes per page; the score folds the scale out of the dot product, $\mathrm{relu}(s\,(q \cdot c))$. Rebuilt from the device's own indexer tensors, the top-512 selection keeps 99.6% of its blocks with int8 keys (fp8: 98.3%; a PCA to 64 dimensions: 76.7%). Together with the split (Section 3.1), block keys fall from 0.75 to 0.20 KiB/token per card (Fig. 3). `-state-cache-dtype f16` halves the GDN state from 54 to 27 MiB per sequence and card.

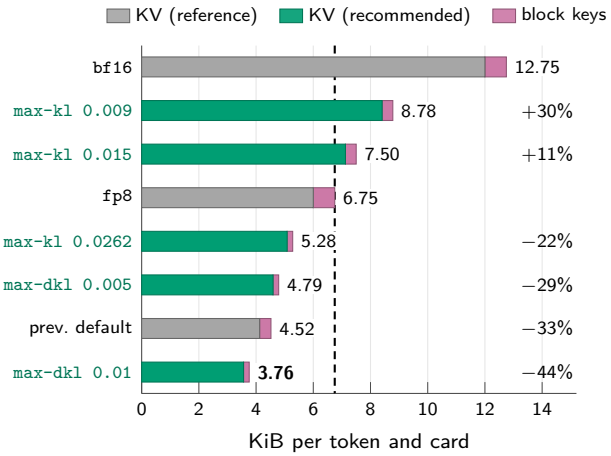


Figure 3: The recommended recipes store 3.76 to 8.78 KiB per token and card, against 6.75 for fp8. Per-token footprint per card by component (attention KV and block keys); percentages are relative to fp8. The attention KV is 0.75×bpw KiB (12 layers, one KV head of 2×256 elements); block keys take 0.75 (bf16), 0.40 (int8), 0.38 (bf16, split) or 0.20 KiB (int8, split). The dashed line marks fp8. The GDN state is per sequence, not per token: 55.24 MiB in f32, 28.24 MiB in f16.

4.4 Tiered precision

A recipe line may name a narrower *demote* layout for the blocks the sweep moves to the store. Sink and tail, read on almost every step, stay in the resident layout (`i8h`, or `fp8` rotated). On demotion an on-device transcode decodes the resident layout exactly and encodes the demote layout with its own store encoder; on page-in and promotion the inverse expands it, so attention still reads one layout per layer. The transcode never rotates, centres or rescales, so both layouts share the layer's basis and V mean. A block is rounded through the demote layout once in its life: a promoted block keeps its origin slot and re-points on re-demotion. Prefix-cache cold entries hold demote-precision bytes.

5 AutoQuant: a measured search

`rad-autoquant` chooses the KV variant of every attention layer for a `-kv-recipe`. It treats the device as the only oracle: each candidate recipe is a run of the engine's KL mode on the long corpus against a bf16-cache reference, and every recipe it returns has been measured, not predicted. Algorithm 1 gives the procedure, Fig. 5 shows what it does on this model and Table 3 what it costs.

5.1 Inputs and outputs

Inputs. (i) A reference recorded by the engine's KL mode with the same model, flags and corpus and

a bf16 cache (`radiance ... -kv-cache-dtype bf16 -kld-record REF -kld-corpus LONG.jsonl`), so that a run's KL is what its cache costs and nothing of the weight quantisation. A narrowed layer's error is paid where attention reaches far back, so the tool refuses a reference whose shortest document is below `-min-context` tokens (default 32 768); ours has 33 998. (ii) The attention layers (`-layers`). (iii) One or more limits (Section 5.3). (iv) The calibration files `-vmean` and `-vpred` (Section 4); the variants that use them join the candidates only when they are given. (v) A work directory (`-work`) that keeps each run's recipe, KL report, per-position rows and log; with `-reuse` a run whose recipe and command line match is read back instead of run. The engine flags after `-` go to every run unchanged; the cache and KL flags belong to the tool and are refused there. A failed run stops the tool with its log's last lines, and no recipe is written. The five recipes of this paper came from one command per limit:

```
rad-autoquant --kld-ref ref-bf16 \
  --layers 3,7,11,15,19,23,27,31,35,39,43,47 \
  --max-kl 0.009 \
  --vmean kv_vmean.txt \
  --vpred kv_vpred.txt \
  --reuse --work aq-work \
  -o max-kl-0.009.kvrecipe \
  -- --model $MODEL --tp 2 --max-num-seqs 2 \
     --max-model-len 36864 $EXTRA
```

looped over `-max-kl 0.0262`, `0.009`, `0.015` and `-max-dkl 0.01`, `0.005`, in that order, all in one work directory.

Output. A recipe file (Section D) whose comments record every measurement the search made (Fig. 4): the uniform runs, the variants screening dropped, the noise floor, the per-layer cost table, every solved round with its room, prediction and measurement, the first pass's answer, the variant order and step costs of the second pass, the chosen run with its KL, top-1, ΔNLL and intervals, each knob run, the knob setting kept and a `# serve with:` line naming the flags the recipe needs. Bytes per token and state bytes per sequence are read from the engine's own KV report in the run's log, not computed from bits.

5.2 Variant space and screening

Variants. A candidate is every layout of the recipe table (Table 2) as the recipe parser reads it: plain, `rotated`, with `-vmean vmean`, with `-vpred vpred`, and the rotated forms of those. Combinations the parser refuses are not variants (`fp8 vmean`: E4M3 V has no group scale; `k6_v4 vpred`: the prediction is made from `k4h`'s int4 K), and one it reads as another is listed once (`k4h_v4 rotated` is `k4h_v4`, whose rotation is implied). That gives 12 variants without calibration files, 21 with `-vmean` and 23 with both; a layer takes `vmean` or `vpred`, never both. A layout added to the table and parser becomes a candidate without a change to the tool. Bits count a group of 32 K and 32 V elements with their scales; rotation and the V mean cost none.

Uniform runs and the base. The search first runs every variant on every layer (23 runs, Fig. 5a). The variant of least mean KL is the *base*: `bf16`, whose KL against the bf16 reference is zero, so every limit down to zero is reachable.

Screening. A variant beaten on its uniform run by another variant of no more bits gets no per-layer runs; at equal bits the lower KL stays. On this model screening drops 16 of the 22 quantised variants and keeps six: `i8h`, `fp8`, `k6_v4` rot+vm, `k5_v4` rot+vm, `k4h_v4 vpred` and `k4h_v2h vpred`. The transforms decide within a layout: rotation with the V mean wins for `k6_v4` (0.0309 against 0.0321–0.0352) and `k5_v4` (0.0328 against 0.0339–0.0374); V prediction wins for `k4h_v4` (0.0373 against 0.0394 with the V mean and 0.0437 plain) and `k4h_v2h` (0.0555 against 0.0833 and 0.1372). `i8h` with the V mean and rotated `fp8` lose to their plain forms at equal bits, and the best `fp8_v4` (rot+vm, 0.0320 at 6.25 bits) loses to `k6_v4` rot+vm at 5.50. Screening is a heuristic, since one layer can differ from the model's average; `-keep-dominated` measures every variant. A dropped variant's uniform recipe still competes for the answer.

5.3 Noise floor and limits

Noise floor. Under a measured limit the base is run once more with `-noise-flags` (default `-max-num-batched-tokens 64`), which changes only the order in which the model computes. Its mean KL, 0.0243 (±0.0026), is the floor of Section 6: the anchor of `-max-dkl` and a scale for a recipe's KL, not a lower bound for KV-only changes (uniform `i8h` measures 0.0008 below it).

Limits. `-max-kl K` admits mean KL $\leq K$; `-max-dkl D` admits mean KL $\leq$ floor $+D$ (0.0293 at $D = 0.005$, 0.0343 at $D = 0.01$); `-max-dnll N` bounds the mean NLL change; `-min-top1 P` bounds top-1 agreement from below. Given together, a recipe must meet every one. Each run's mean is reported with a 95% interval from its per-position rows (batch means with each document a stratum, Student's t). With `-strict` every limit is checked at the bound on the bad side, for `-max-dkl` the run's upper bound less the floor's lower bound. On four documents the half-width is about 0.0025 at `fp8`, as large as the steps between layouts, so a limit is a statement about this corpus.

5.4 Per-layer costs, solve and verify

Costs. Each layer alone in each kept variant, the rest at the base, gives $L(K-1)$ runs for L layers and K kept variants (the base included): $12 \times 6 = 72$ here (Fig. 7). Let b_v be the bits of variant v and $d_{\ell v}$ the mean KL of that run above the base, clamped at zero. For an absolute limit K the solver picks

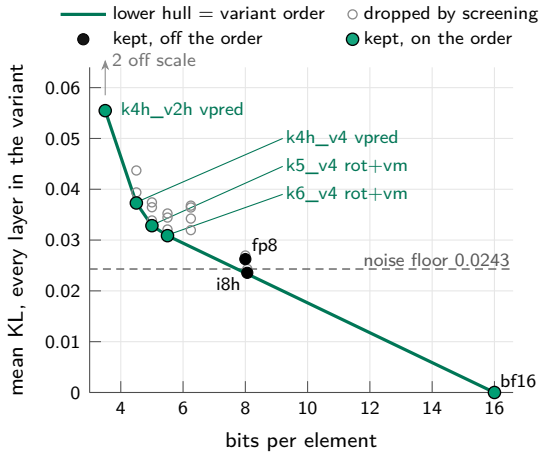
$$\min_{a:L\to V} \sum_{\ell} b_{a(\ell)} \quad \text{s.t.} \quad \mathrm{KL}_{\mathrm{base}} + \sum_{\ell} d_{\ell,a(\ell)} \leq K \qquad (6)$$

exactly, as a knapsack over bits spent: a dynamic programme over the bit budget whose table is read for the

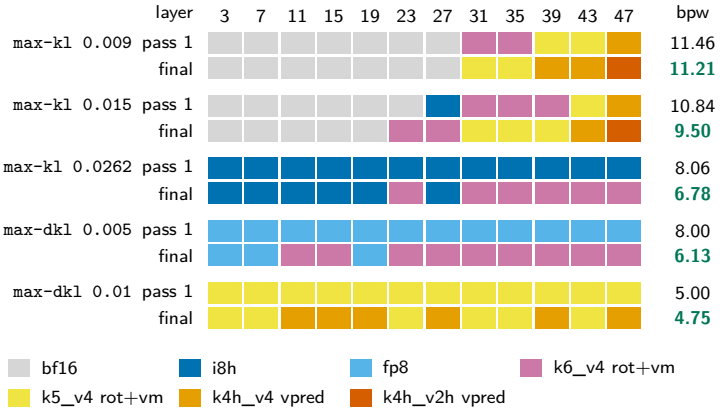
```
# rad-autoquant: a KV layout per attention layer, the fewest
#   bits measured within a mean KL of 0.009
# reference ref-bf16, 4 documents, the shortest 33998 tokens
# every layer bf16: mean KL 0.000000 +- 0.000000, top-1 1.0000
#   +- 0.0000, -0.024306 above the floor, dNLL +0.0000 (the base)
# every layer i8h: mean KL 0.023552 +- 0.002552, top-1 0.9477
#   +- 0.0064, -0.000755 above the floor, dNLL -0.0024 +- 0.0046
#   [... 21 more uniform runs ...]
# noise floor, every layer bf16 with --max-num-batched-tokens 64:
#   mean KL 0.024306 +- 0.002561, top-1 0.9474 +- 0.0059, ...
# no per-layer runs: k6_v4 (0.034429 at 5.50 bits;
#   k6_v4 vmean: 0.032083 at 5.50)    [... 15 more drops ...]
# each layer alone in a variant, the rest the base:
#   mean KL above the base
#   layer        i8h        fp8  k6_v4+rotated+vmean  ...
#       3  2.359e-02  2.384e-02            2.469e-02  ...
#      27  6.077e-03  6.723e-03            8.156e-03  ...
#      47  1.411e-04  2.812e-04            2.730e-04  ...
# solved 1: room 0.009000 above the base, predicted mean KL
#   0.008928, measured 11.4583 bits, mean KL 0.005130 +- 0.000516
```

```
# pass 2: the variant order (...on the lower hull of their uniform
#   runs' mean KL): bf16 > k6_v4 rotated vmean > k5_v4 rotated
#   vmean > k4h_v4 vpred > k4h_v2h vpred; off it: i8h, fp8
# pass 2 from verify 1: each layer alone one step narrower:
#      31  k6_v4 rotated vmean -> k5_v4 rotated vmean  4.001e-04
#      47  k4h_v4 vpred -> k4h_v2h vpred  4.360e-04
# pass 2 solved 1: room 0.003870 above verify 1, predicted mean KL
#   0.007005, measured 11.2083 bits, mean KL 0.007163 +- 0.000691
# chosen: 11.2083 bits, mean KL 0.007163 +- 0.000691, top-1 0.9702
#   +- 0.0041, dNLL +0.0008 +- 0.0020 (pass 2 verify 1)
# with --qsa-bkey-dtype i8: ... mean KL 0.025890, above the limits
# with --state-cache-dtype f16: ... mean KL 0.027716,
#   above the limits
# with --qsa-bkey-split on: not run, exact (...): block keys
#   0.75 KiB a token, 0.38 KiB split
# knobs: --qsa-bkey-split on, ...: 8.78 KiB a token
# serve with: --kv-cache-dtype fp8 --kv-recipe
#   max-kl-0.009.kvrecipe --qsa-bkey-split on
vmean kv_vmean.txt
vpred kv_vpred.txt
layer 3 bf16                 (... layers 7–27 bf16)
layer 31 k5_v4 rotated vmean (also 35)
layer 39 k4h_v4 vpred        (also 43)
layer 47 k4h_v2h vpred
```

Figure 4: The recipe is its own lab notebook: every number behind the answer is written into it as a comment. Excerpt of `max-kl-0.009.kvrecipe`, the search answer behind the default recipe (which keeps layer 47 in bf16); long comment lines are wrapped and abridged (..., bracketed notes), and only 3 of 12 rows of the per-layer table and 2 of 12 second-pass steps are shown. The last lines are what the engine parses.



(a) Uniform runs and the second pass's variant order.



(b) Per-layer assignment of the five answers, first pass and final.

Figure 5: The second pass narrows from the best first-pass recipe along the lower hull of the uniform runs. (a) The 23 uniform runs of `rad-autoquant` (long corpus). Screening keeps the filled points; the second pass steps a layer only along the hull (green), so `i8h` and `fp8`, which lie above it, step to `k6_v4` rot+vm. Two `k4h_v2h` variants (plain 0.137, `vmean` 0.083) are off scale. (b) Variant per attention layer of each limit's first-pass answer and final answer (Table 5), with bits per element. Under the absolute limits the first pass already narrows the late layers and the second pass narrows them one step further; at `-max-kl 0.0262` and `-max-dkl 0.005` the first pass stops at a uniform 8-bit layout and the second pass moves 6 to 9 of 12 layers to `k6_v4` rot+vm, keeping `i8h` or `fp8` mostly on the early layers.

fewest bits within the KL room. Under `-max-dkl` the room is floor $+D - \mathrm{KL}_{\mathrm{base}}$.

Measure and refit. Costs do not add. One quantised early layer costs as much as all of them: `i8h` on layer 3 alone measures 0.0236, every layer in `i8h` 0.0236. Every solution is therefore run on the device (`verify` runs). If any check fails, the room the solver may use becomes the solution's prediction times allowed over measured excess for the tightest failing check, kept strictly below the rejected prediction, and the problem is solved again, at most `-verify` times (default 6). A solution with no fewer bits than a recipe already measured within the limits is not run. ΔNLL and top-1 are checks on measured recipes, not summed costs, because a single layer's ΔNLL is about as noisy as the limit. The answer is the recipe of fewest bits among *all* recipes measured within the limits (uniform, single-layer or solved), so it is never worse than the best uniform variant.

5.5 Second pass

Per-layer costs measured from the bf16 base cannot see narrowing that starts from a uniform recipe already within the limit. At `-max-kl 0.0262` the first pass answers uniform `i8h` (8.06 bpw): the next solution has no fewer bits and is not run. The second pass takes the best recipe measured within the limits after the first pass as a new base B_2 and moves each layer alone one step narrower, the rest as in B_2: at most L runs. The

Algorithm 1: AutoQuant (`rad-autoquant`)

Input: reference R, layers L, variants V, limits Λ, rounds N, knobs
Output: recipe with every measurement as comments

foreach $v \in V$ **do** $U_v \leftarrow$ `Run`(all L in v) `// uniform`
$\beta \leftarrow \arg\min_v \mathrm{KL}(U_v)$ `// base, bf16`
$S \leftarrow \{v : \nexists w\text{: } b_w \le b_v,\ \mathrm{KL}(U_w) < \mathrm{KL}(U_v)\}$
$f \leftarrow$ `Run`(β with noise flags); resolve Λ against f
foreach $\ell \in L,\ v \in S \setminus \{\beta\}$ **do**
  $d_{\ell v} \leftarrow \max(0, \mathrm{KL}(\texttt{Run}(\ell{:}v)) - \mathrm{KL}(\beta))$
$A \leftarrow$ fewest bits of all runs within Λ
$A \leftarrow \texttt{Fit}(\beta, d, \Lambda, A)$ `// pass 1`
$O \leftarrow \texttt{LowerHull}(\{(b_v, \mathrm{KL}(U_v)) : v \in S \cup \{\beta\}\})$
if $A \neq \beta$ **then** `// pass 2`
  foreach $\ell \in L$ **do** $s_\ell \leftarrow$ next in O after $A(\ell)$;
    $e_\ell \leftarrow \mathrm{KL}(\texttt{Run}(A,\ \ell{:}s_\ell)) - \mathrm{KL}(A)$
  $A \leftarrow \texttt{Fit}(A, \{A(\ell){:}0,\ s_\ell{:}e_\ell\}, \Lambda, A)$
foreach $k \in \{bkey_i8, state_f16\}$ **do** `Run`($A + k$)
if *both within* Λ **then** `Run`(A + both)
keep the setting within Λ of fewest bytes/token; add `bkey_split` (exact, not run)
return recipe A, knobs, comments, serve line
Function `Fit`(B, c, Λ, A):
  room $\leftarrow$ limit $-$ KL(B)
  for $r \leftarrow 1$ **to** N **do**
    $a \leftarrow \texttt{Solve}(c, \text{room})$ `// Eq. (6)`
    if bits(a) $\ge$ bits(A) **then break**
    $m \leftarrow$ `Run`(a); **if** m *within* Λ **then return** a
    room $\leftarrow \min(\text{pred}(a) \cdot \frac{\text{allowed}}{\text{excess}},\ \text{pred}(a) - \epsilon)$
  return A

step is the next variant in the *variant order*: the kept variants by bits, keeping only those on the lower convex hull of uniform KL over bits (Fig. 5a). A variant above the line between its neighbours spends KL for almost no bits: `fp8` (8.00 bpw, 0.0262) beside `i8h` (8.06, 0.0236) saves 0.06 bits a step, and `i8h` lies 0.0002 above the line from `bf16` to `k6_v4` rot+vm. The order on this model is therefore bf16 > `k6_v4` rot+vm > `k5_v4` rot+vm > `k4h_v4 vpred` > `k4h_v2h vpred`, and a layer in `i8h` or `fp8` steps to `k6_v4` rot+vm. Each layer then has two candidates, its variant in B_2 (cost zero) and its step (measured cost above B_2), and the same fit loop solves, measures and tightens under the same limits. The pass is skipped when B_2 is the first base, and is not repeated from its own answer. Figure 5b shows the first- and second-pass answers of all five limits; Section 7 discusses each.

5.6 Global knobs

Three options act on the whole model rather than per layer. The chosen recipe is measured once more with int8 block keys (`bkey_i8`) and with the f16 recurrent state (`state_f16`), each alone, and with both only when each alone stayed within every limit. Of the answer and the knob settings within the limits, the one of fewest bytes per token is kept, then of fewest state bytes per sequence; a tie keeps fewer knobs. Both knobs perturb every layer (block keys move the selection, the state every token), so they fit only where a recipe is already at the floor (Section 7). The block-key split (`bkey_split`) is exact: the ranks merge their local top-k lists into the unsplit selection byte for byte, so the KL rows are the same. It is never run, always kept at tensor parallelism 2, and its halved block keys are counted in the bytes per token and written into the serve line.

5.7 Cost and run reuse

A search costs $C + L(K-1)$ runs, plus under a measured limit one noise run, at most N solved recipes, at most $L + N$ for the second pass and at most three knob runs. Every run has a name that identifies its recipe: `uniform_VARIANT`, `layerN_VARIANT`, `noise_BASE_HASH`, `verify_HASH` and `step_layerN_VARIANT_HASH` (hash of the recipe text) and `knob_KNOBS_HASH`. Its command line, including content hashes of the V-mean and V-prediction files, is stored beside it, so with `-reuse` a search at another limit reruns only its solved recipes, the second-pass steps from a B_2 not measured before and its knob runs. A change of engine flags, reference or calibration files reruns everything; the engine binary is not hashed.

Table 3 counts the runs in the logs. Each limit evaluates 112 or 113 recipes. The first limit measured 58 new runs (the six uniform runs and 36 per-layer runs of the variants new in this version, the second pass and the knobs) and read 54 back from the previous version's work directory; it took 4 222 s. Every further limit measured 16 or 17 new runs, 12 of them second-pass steps, in 1 120–1 219 s, about 70 s per run. The first-pass-only version took 5 075 s for its first limit (70 new runs) and 210–212 s for each further one (3 new runs), so the second pass adds about a quarter of an hour per limit.

5.8 From one pass to two

The previous version of the tool had 17 variants (no `k5_v4`, no `vpred`) and no second pass; its answers are the first rows of Table 6. Because its costs were measured only from the bf16 base, it answered the extremes: bf16-heavy recipes under the absolute limits and a uniform layout (`i8h`, `fp8`, `k6_v4` rot+vm) at `-max-kl 0.0262` and both `-max-dkl` limits. Recipes between them (the fp8-equivalent recipe A, probes below 5.5 bpw) were found by hand. The new variants alone move the first pass little (Table 3: −0.08 and −0.04 bits at `-max-kl 0.009` and `0.015`, none at `0.0262` and `-max-dkl 0.005`, −0.5 at `-max-dkl 0.01` through `k5_v4` rot+vm); the second pass does most of the work, −0.25 to −1.87 bits from the first pass, and recovers recipe A's structure at `-max-kl 0.0262` without help.

5.9 Relation to ModelOpt AutoQuant KV

ModelOpt's AutoQuant KV [11] also assigns per-layer KV formats by KL under a budget. Ours differs in three ways. Every answer is measured on the target device, with the deployed kernels and tensor parallelism,

Table 3: The second pass costs 13–14 runs per limit and removes 0.25 to 1.87 bits per element from the first-pass answer. Bits per element of the previous, first-pass-only tool (17 variants, no `k5_v4/vpred`; Table 6), of the current tool's first pass and of its final answer (Table 5), and the runs of each search as counted in its log: new/reused, in the order the limits ran. "verify" are pass-1 solutions; "pass 2" is per-layer steps + solutions. Wall time is the whole invocation on the two cards.

	bpw			runs (new/reused)							
limit	previous tool	pass 1	final	uniform	noise	per-layer	verify	pass 2	knobs	total new/reused	wall (s)
`-max-kl 0.0262`	8.06	8.06	6.78	6/17	0/1	36/36	–	12+1	3	58/54	4 222
`-max-kl 0.009`	11.54	11.46	11.21	0/23	0/1	0/72	1	12+1	2	16/96	1 159
`-max-kl 0.015`	10.88	10.84	9.50	0/23	0/1	0/72	1	12+1	2	16/96	1 167
`-max-dkl 0.01`	5.50	5.00	4.75	0/23	0/1	0/72	–	12+2	3	17/96	1 219
`-max-dkl 0.005`	8.00	8.00	6.13	0/23	0/1	0/72	–	12+2	2	16/96	1 120

as full-vocabulary KL against a bf16-cache run of the same model, and the measurement, not a sum of per-layer scores, decides; the summed costs are used only to propose candidates. Limits can be stated relative to a measured noise floor (`-max-dkl`), with intervals and an optional strict bound. And a second pass searches around the best measured recipe, which a score computed from the unquantised base cannot reach.

6 Evaluation methodology

Reference and metrics. The engine's KL mode records the full next-token distribution of the model with a bf16 KV cache at every scored position, teacher-forced on fixed text, and scores a candidate against it: $\mathrm{KL}(p_{\mathrm{ref}} \| p_{\mathrm{cand}})$ at temperature 1, computed exactly over the full vocabulary (mean, median, p99), top-1 agreement, and the change in negative log-likelihood of the actual next token, ΔNLL (log of the perplexity ratio). The reference is the *same quantised model*, so the numbers isolate what KV compression changes; the original bf16 checkpoint was not measured.

Corpora. *long*: four documents of about 34K tokens, the last ∼4 000 positions of each scored (16 380). *recall*: a needle-style corpus of four documents, a ∼2 000-token passage repeated ∼26K tokens later, only the repeat scored. *multi-round*: four conversations of four turns, each pasting ∼6 000 tokens of a file with a question and answered by the model itself (bf16, greedy), then a fifth turn asking about the first file; only that answer is scored (256 tokens per conversation, 1 020 positions).

The noise floor. The noise floor is the KL of a bf16-cache run at prefill chunk 64 instead of 2 048: **0.0243** (95% half-width 0.0026, documents as strata). A configuration's KL *above the floor* is the paired per-position difference between that configuration at chunk 64 and the floor run. Pairing narrows the position-block bootstrap intervals two to three times, to about ±0.001. The search's `-max-dkl` and its log report the simpler, unpaired mean KL minus the floor; where tables use that, they say so.

Uncertainty. Intervals are 95% block bootstraps over 256-position blocks. Because the KL per position is already exact over the vocabulary, the remaining variance is between positions and documents (a Rao-Blackwellised estimator would not reduce it). Position blocks understate the variance between documents: per document, the paired difference has a standard error of 0.0006 (`i8h`) to 0.0028 (`k4h_v4` with V mean), so with four documents a t-interval is three to six times wider than the block bootstrap. The sign of every difference reported above the floor is the same in all four documents; finer distinctions need a larger corpus.

Proxy. For screening, numpy reproductions of the quantisers run on query, key and value tensors dumped from the device (all 12 layers, one KV head; a code prefill for calibration, a prose prefill for measurement); the metric is the relative error of the dense causal attention output. It ranks methods but is not proportional to KL and does not model the selection. Proxy numbers are marked as such.

7 Results

7.1 The Pareto frontier

Figure 6 places every measured configuration by footprint and mean KL. Uniform layouts form a staircase that is dominated almost everywhere by mixed recipes, and the search's answers lie on the frontier from 3.76 to 8.78 KiB/token. Below the noise floor only recipes with bf16 on the early attention layers exist; above it, the frontier is flat between 4.5 and 6.4 KiB/token (KL 0.0243 to 0.0328), which is where the choice of limit matters most.

7.2 Uniform layouts

Table 4 reports the long corpus. All 8-bit layouts are within +0.003 of the floor and their ΔNLL is indistinguishable from zero; their KL above the floor is small but, except for `i8h` with int8 block keys, not zero. fp8's KL is entirely storage rounding: a bf16 cache whose values are rounded through E4M3 gives position-for-position identical results.

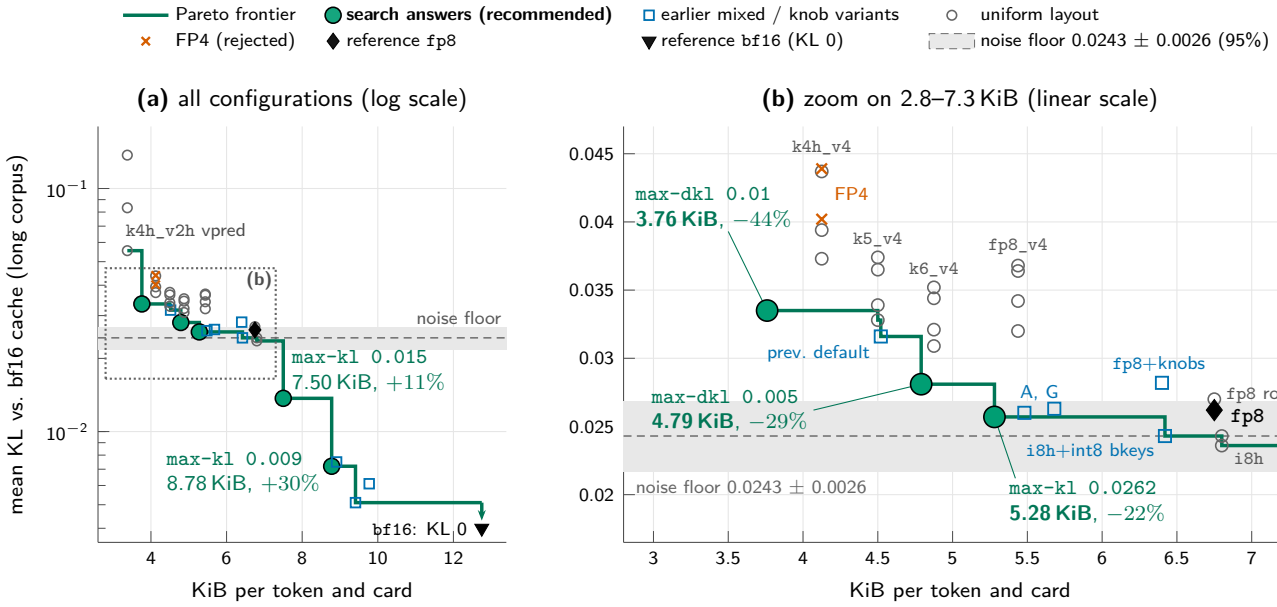


Figure 6: The search's answers form the Pareto frontier from 3.76 to 8.78 KiB per token (long corpus, mean KL against a bf16-cache run). (a) All measured configurations on a log scale; (b) the crowded region on a linear scale. The step line is the Pareto frontier: the `rad-autoquant` answers (green), uniform layouts at the ends and `i8h`, and earlier mixed recipes. Open circles stacked in one column are one layout with and without rotation, V mean or V prediction. Points sit at their measured footprint with the knobs they were measured with; for uniform layouts (bf16 block keys) that is 0.75 (bpw + 1) KiB. The grey band is the noise floor with its 95% interval. Everything to the left of fp8 below its KL is an improvement; the search reaches fp8's KL 22% smaller and stays within floor +0.01 at 44% smaller.

The transforms matter more than a bit of key precision. The V mean lowers the KL above fp8 by 25–48% on every integer-V layout; rotation helps 4-bit keys, and `fp8_v4` together with the V mean; V prediction lowers the KL above the floor by a further 22% (`k4h_v4`, +0.0162 → +0.0126) and 48% (`k4h_v2h`, +0.0567 → +0.0296) compared with the V mean. `k5_v4` rot+vm (+0.0084 at 5.0 bpw) fills the gap between `k6_v4` (+0.0064 at 5.5) and `k4h_v4` without a significant likelihood change on any corpus. Both halves of the row matter: lowering only the keys from 6 to 4 bits (`k6_v4` rot+vm to `k4h_v4` with V mean) raises the KL above the floor 2.5 times, while in the proxy 3-bit values cost more than 5-bit keys save (`k5_v4`: 0.031, `k6_v3`: 0.066, `k6_v4`: 0.028), and with 2-bit values more key bits buy nothing. Only 2-bit values lose likelihood significantly, even with `vpred`. The full floor-corrected table of all 30 configurations is in Section E.

7.3 Per-layer sensitivity

Figure 7 shows the KL of quantising a single attention layer with the rest in bf16: sensitivity falls by about 170× from the first attention layer to the last (`i8h` alone costs 0.0236 on layer 3 and 0.00014 on layer 47), and the ordering of the variants is the same on every layer. Nearly all of the KV's KL comes from layers 3–23: bf16 on the first six attention layers with `i8h` on the rest measures 0.0061, the reverse 0.0239. The costs are far from additive (one early layer costs as much as all twelve), which is why the search measures its answers.

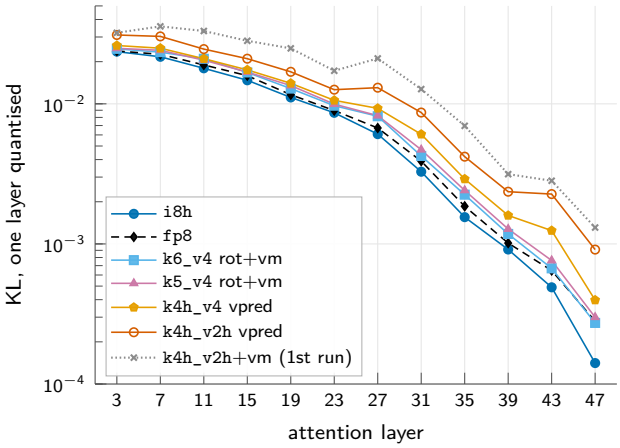


Figure 7: Sensitivity falls ∼170× with depth. KL against the bf16 reference when only one attention layer is quantised (log scale; per-layer table of the current search run, plus `k4h_v2h` with V mean from the first run). Early layers need 8 bits or more; late layers tolerate 2-bit values.

7.4 Search answers

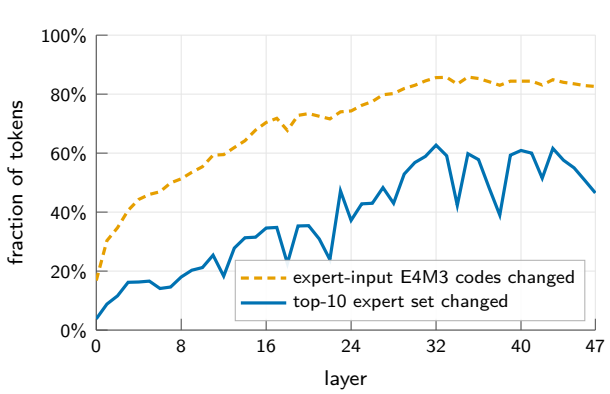
Table 5 lists the five `rad-autoquant` answers of the current run and Fig. 5b their layer assignments. The second pass improves every limit. At `-max-kl 0.009` it narrows the last five layers of the first-pass answer (11.46 bpw, KL 0.0051) to 11.21 bpw at KL 0.0072; at 0.015 it moves from 10.84 bpw (KL 0.0075) to 9.50. At `-max-kl 0.0262` it starts from uniform `i8h` (8.06 bpw, KL 0.0236); every single step costs at most 0.0013, and the pass finds `i8h` on layers 3–19 and 27 with `k6_v4` rot+vm on 23 and 31–47 (6.78 bpw, KL 0.0249), the recipe we had first found by hand (Table 6). At

Table 4: Uniform layouts, long corpus (16 380 positions). KL above the noise floor (0.0243) is paired per position at chunk 64, 95% interval in parentheses. KiB/token per card including block keys, where measured. 2-bit values are the only layouts with a significant likelihood loss (bold).

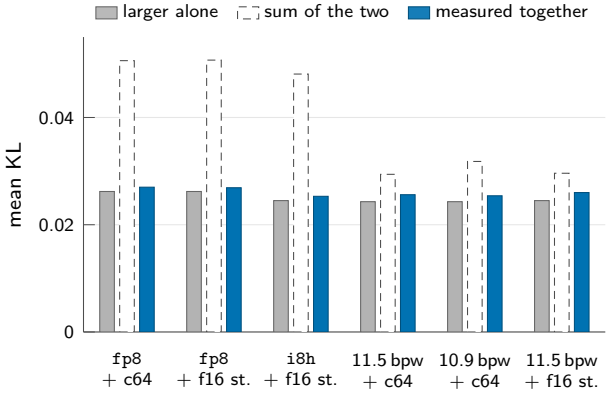
configuration	bpw	KiB/token	mean KL	KL above floor (95%)	top-1 (%)	ΔNLL (95%)
`bf16`	16	12.75	0	–	100.0	0
`i8h`	8.06	6.80	.0236	+.0016 (.0005,.0028)	94.77	−.0024 (−.0064,.0015)
+ int8 block keys	8.06	6.42	.0243	+.0007 (−.0001,.0016)	94.81	
`fp8`	8	6.75	.0262	+.0027 (.0015,.0038)	94.35	+.0010 (−.0028,.0052)
+ int8 block keys + f16 state	8	6.40	.0282	+.0037 (.0026,.0050)	94.19	−.0008 (±.0056)
`fp8_v4` rot+vm	6.25		.0320	+.0075 (.0060,.0091)	94.03	+.0011
`k6_v4` rot+vm	5.5	4.88	.0309	+.0064 (.0050,.0078)	94.09	+.0017 (−.0032,.0064)
+ int8 block keys + f16 state	5.5	4.52	.0316	+.0075 (.0060,.0092)	93.83	+.0026 (±.0051)
`k6_v4`	5.5		.0344	+.0109 (.0096,.0124)	93.66	+.0008
`k5_v4` rot+vm	5.0		.0328	+.0084	93.74	−.0006
`k5_v4`	5.0		.0374	+.0147	93.26	+.0043
`k4h_v4` vpred	4.5		.0373	+.0126	93.24	+.0054
`k4h_v4` + V mean	4.5		.0394	+.0162 (.0136,.0185)	93.20	+.0028 (−.0022,.0082)
`k4h_v4`	4.5		.0437	+.0217 (.0193,.0243)	93.08	+.0040
`k4h_v2h` vpred	3.5		.0555	+.0296	91.87	**+.0083**
`k4h_v2h` + V mean	3.5		.0833	+.0567 (.0503,.0627)	89.90	**+.0132** (.0063,.0213)
`k4h_v2h`	3.5		.1372	+.1113 (.0987,.1222)	87.73	**+.0345**



(a) How a small numerical perturbation becomes the noise floor.



(b) Routing flips per layer.



(c) Two perturbations combined.

Figure 8: A small numerical perturbation becomes the noise floor of the quantised MoE model. (a) The propagation chain. (b) A different prefill order alone flips most routing decisions by mid-depth: recomputing the bf16-cache model at prefill chunk 64 instead of 2 048, per layer (7 008-token prefill). (c) Perturbations saturate instead of adding: two perturbations applied together (c64 = prefill chunk 64, f16 st. = f16 recurrent state) measure close to the larger of them, not to their sum.

`-max-dkl 0.005` (KL ≤ 0.0293) the pass starts from uniform fp8; its first answer (5.71 bpw) measured 0.0306, above the limit, and the re-solve settled at 6.13 bpw. At `-max-dkl 0.01` it starts from uniform `k5_v4` rot+vm (5.00 bpw, KL 0.0328); 4.67 bpw measured just above the limit, 4.75 bpw within it.

The knobs behave as predicted by the floor. On the two near-lossless recipes, int8 block keys lift the KL from 0.0072 to 0.0259 and from 0.0137 to 0.0272, and the f16 state to 0.0277 and 0.0287, so only the exact split is kept. On the recipes already at the floor, int8 block keys cost nothing measurable or even lower the KL (`dkl 0.005`: 0.0287 → 0.0281), and on the most compact recipe both knobs together fit (0.0343 → 0.0335).

The previous tool version also probed recipes below 5.5 bpw by narrowing late layers of the uniform `k6_v4` rot+vm recipe by hand, all with both knobs: 5.08 bpw 0.0346, 5.17 bpw 0.0345, 5.0 bpw 0.0356/0.0364, 4.75 bpw 0.0384, none reliably under the floor +0.01. With `k5_v4`, `vpred` and the second pass, the current search finds 4.75 bpw at 0.0335.

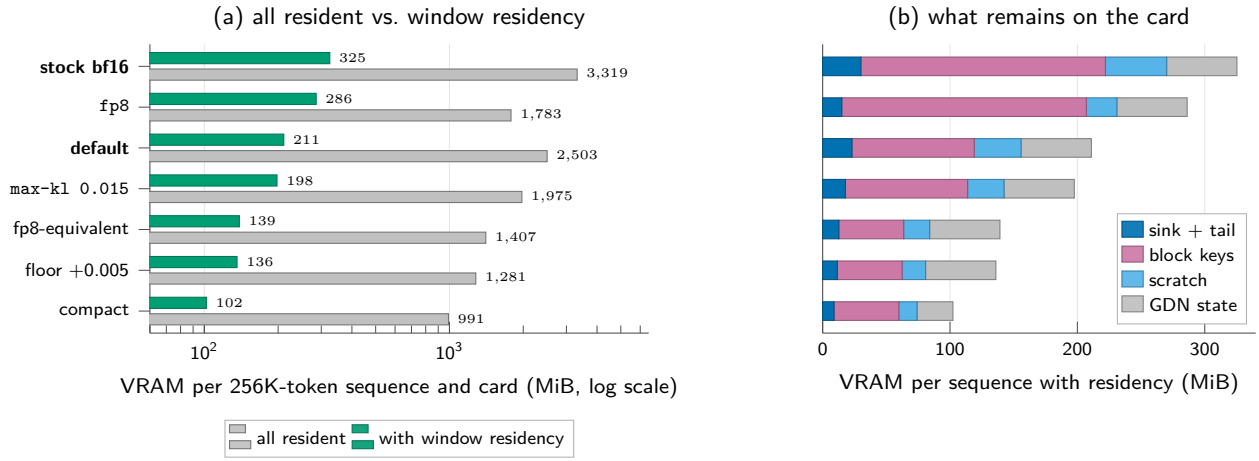


Figure 9: Window residency removes 90–97% of a long sequence's VRAM; what remains is mostly block keys and recurrent state. One sequence at the maximum length of 256K tokens on one card (tensor parallelism 2), computed from the measured bytes per token. (a) The whole cache resident (grey) against window residency (green), log scale. (b) The residency footprint by component. Sink and tail are 2 560 tokens of attention KV. The block keys cover all tokens, halved by the split and again by int8. The scratch pages hold one decoding sequence's selected cold blocks. The GDN state is 55 MiB in f32 and 28 MiB in f16. Only the block keys grow with the context.

Table 5: Search answers (`rad-autoquant` with the second pass), long corpus, with the knobs kept; the recipes are listed under Table 1. "KL−floor" is mean KL minus the floor 0.0243 (unpaired), as used by `-max-dkl`; "chunk 64 above floor" is the paired per-position measure of the other tables. The first-pass answers and the cost of each search are in Table 3.

limit	bpw	KiB/ token	mean KL	KL− floor	ΔNLL	top-1 (%)	chunk 64 above floor	recall KL	multi- round KL
`-max-kl 0.009`	11.21	8.78	.0072	−.0171	+.0008	97.02	+.0029	.0014†	.0020
`-max-kl 0.015`	9.50	7.50	.0137	−.0106	+.0002	95.86	+.0036	.0043	.0027
`-max-kl 0.0262`	6.78	5.28	.0257	+.0014	−.0016	94.57	+.0019	.0107	.0036
`-max-dkl 0.005`	6.13	4.79	.0281	+.0038	+.0022	94.10	+.0061	.0115	.0038
`-max-dkl 0.01`	4.75	3.76	.0335	+.0092	+.0008	93.53	+.0099	.0125	.0051

7.5 VRAM per sequence against the stock cache

Quantisation and window residency save VRAM in different places. Quantisation shrinks every token of the cache. Residency removes the cold middle from the card altogether: per sequence it keeps only the sink and the tail (2 560 tokens), the block keys of all tokens, the scratch pages that hold the step's selected cold blocks, and the recurrent state. Table 7 and Fig. 9 combine both for one sequence at the model's maximum length of 256K tokens (262 144) on one card, against the engine's stock configuration (bf16 cache, bf16 block keys, f32 state, everything resident).

The figures are computed from the measured bytes per token of each configuration (Table 1). The attention part of a token is the KiB/token minus its block keys, which take 0.75 (bf16), 0.375 (bf16, split), 0.40 (int8) or 0.20 KiB (int8, split) per token and card.

- *Without residency*: tokens × KiB/token, plus the state.
- *With residency*: 2 560 resident tokens of attention KV, plus the block keys of all tokens, plus the scratch pages of one decoding sequence ($513 \times (1 + n_{\text{spec}})$ blocks, $n_{\text{spec}} = 1$), plus the state.

With residency, the block keys are the only per-sequence item that still grows with the context: 0.2 to 0.75 MiB per thousand tokens. Quantisation then pays off mainly in host memory, which bounds the number of sessions (Section 7.10), and in PCIe traffic. In VRAM it still shrinks the sink, the tail and the scratch, while int8 block keys, the split and the f16 state become the dominant levers. The compact recipe has all three and needs 102 MiB per 256K sequence, 32 times less than stock; the default needs 211 MiB, 16 times less.

7.6 Tiered precision

Table 8 keeps `i8h` resident and narrows only the demoted middle. A 3.5-bit middle costs less than a third of a uniform 3.5-bit cache on the long corpus; a 4.5-bit middle stays within the recomputation noise on the multi-round corpus and costs +0.005 KL on the others, while a 3.5-bit middle shows a significant likelihood loss on the multi-round corpus. Tiered precision was measured only with an `i8h` resident layout, through a debug emulation that applies the round trip in place to exactly the blocks the sweep would demote; residency on equals it byte for byte.

Importance-driven demotion. We also let each demoted block take one of two layouts, `k4h_v4` or `k4h_v2h`, by how often the indexer selected it while resident (an on-device counter after the selection; a fraction F of

Table 6: Earlier mixed recipes: first-pass search answers of the previous tool version (17 variants, no `k5_v4`/`vpred`, no second pass) and hand-found fp8-equivalent probes, long corpus. KL above the floor is paired at chunk 64 where given. All are dominated by Table 5.

recipe	bpw	KiB/token	mean KL	KL above floor	top-1 (%)
bf16 L3–27, `k6_v4` rot+vm L31–43, `k4h_v4`+vm L47 (search, `-max-kl 0.009`)	11.54	9.41	.0051	+.0013	97.45
bf16 L3–23, `i8h` L27, `k6_v4` rot+vm L31–43, `k4h_v4`+vm L47 (`-max-kl 0.015`)	10.88	8.91	.0075	+.0011	96.87
`fp8` everywhere, int8 block keys, f16 state (search, `-max-dkl 0.005`)	8.00	6.40	.0282	+.0037	94.19
`k6_v4` rot+vm everywhere, int8 block keys, f16 state (search, `-max-dkl 0.01`; previous default)	5.50	4.52	.0316	+.0075	93.83
A: `i8h` L3–23, `k6_v4` rot+vm L27–47	6.78	5.84	.0255	+.0030	94.44
A + int8 block keys (fp8-equivalent by hand)	6.78	5.48	.0260	+.0020	94.47
A + f16 state	6.78	5.84	.0268		94.44
G: `i8h` L3–19, `k6_v4` rot+vm L23–47	6.57	5.68	.0263	+.0036	94.39
H: `i8h` L3–15, `k6_v4` rot+vm rest	6.35		.0266		94.38
F: `i8h` L3–11, `k6_v4` rot+vm rest	6.14		.0277		94.28
B: `i8h` L3–23, `k4h_v4`+vm L27–47	6.28		.0301		94.13
E: `i8h` L3–15, `k4h_v4`+vm rest	5.69		.0338		93.50
D: `i8h` L3–11, `k4h_v4`+vm rest	5.39		.0345		93.64
C: `i8h` L3–23, `k4h_v2h`+vm rest	5.78		.0482		92.41
bf16 L3–23, `i8h` L27–47	12.03		.0061	+.0000	97.12
`i8h` L3–23, bf16 L27–47	12.03		.0239		94.77

Table 7: With window residency, a sequence at the maximum length of 256K tokens needs 102–325 MiB of VRAM per card instead of 3 319 MiB. Computed per sequence and card (tensor parallelism 2) from the measured KiB/token; residency assumes one draft token. Residency was verified byte-identical on the device, with the MTP drafter, for `fp8`, `fp8` with an f16 state, the default and the compact recipe with all its knobs (Section C); the other rows follow from their bytes per token and were not run with residency. Measured session counts are in Table 12.

		VRAM per sequence (MiB)		
configuration	KiB/ token	all resident	with residency	vs stock
stock: `bf16`	12.75	3 319	325	−90%
`fp8`	6.75	1 783	286	−91%
default: near-lossless	9.56	2 503	211	−94%
`max-kl 0.015`	7.50	1 975	198	−94%
fp8-equivalent	5.28	1 407	139	−96%
floor +0.005	4.79	1 281	136	−96%
compact	3.76	991	**102**	−97%

blocks gets the wide layout). At $F = 0.75$ the long-corpus KL matches a uniform `k4h_v4` middle (0.0279 vs 0.0284) with 6% fewer PCIe bytes per paged block; at $F = 0.5$ it lands halfway. A control that picks the wide blocks by a hash of their position is, however, as good as the selection count on all three corpora (count minus hash at $F = 0.5$: long −0.0009, recall +0.0024, multi-round −0.0002, all intervals containing zero). The gain comes from mixing layouts, not from the importance signal, and the host store does not shrink because a slot is sized by the wide layout. The mechanism was not adopted.

7.7 Anatomy of the noise floor

A bf16-cache run that differs from the reference only in prefill chunk size measures a mean KL of 0.0243 (top-1 94.74%); one with only an f16 recurrent state, 0.0245. With dense attention (1 800-token documents, where QSA selects every block) the same comparison gives 0.0456, so the floor does not come from the selection. A dense, unquantised Qwen3.5-2B in bf16 shows 0.00018 (top-1 99.43%), about 130 times less.

Propagation. Comparing the two bf16 runs layer by layer (Fig. 8b), the top-10 expert set changes for 3.8% of tokens at layer 0, 16% at layer 3 and about 60% from layer 30 on; expert-input E4M3 codes change from 17% to 85%. Router logits are stored in bf16: at the tenth logit (≈ -4.5) one bf16 step is 0.031, and 20–28% of tokens have an exact tie between the tenth and eleventh expert, another 24–35% a one-step margin. In early layers flips occur almost only there (layer 0: 10.5% of ties, 0.0% of margins of four or more steps). Figure 8a summarises the chain.

Removing ties does not remove the floor. Computing router logits and top-k in f32 lowers the floor only to 0.0239, and leaves every layout's gap to it unchanged within the standard error (Table 14 in Section E); switching to the f32 router itself moves the model by 0.0252. Ties are where the perturbation shows first, not its cause: the model amplifies any perturbation through its discrete decisions (expert routing, dynamic activation quantisation) over 48 layers until it saturates.

Saturation. Perturbations do not add; combinations land on the larger of the two (Fig. 8c). A KV-only perturbation applied late in the network measures far

Table 8: Tiered precision: resident `i8h`, demoted middle in the given layout, residency on (through the emulation). A `k4h_v4`+V-mean middle saves 44% of host memory at no significant likelihood cost; a 2-bit middle loses likelihood on the multi-round corpus (bold). Host KiB per demoted block, both ranks; "above floor" is paired at chunk 64 (long corpus).

demoted middle	host KiB/block	long: KL / top-1 / ΔNLL	recall: KL / top-1	multi-round: KL / top-1 / ΔNLL (95%)	above floor
`i8h` (uniform)	52.5	.0236 / 94.77 / −.0024	.0103 / 99.77	.0032 / 98.63 / −.0016 (−.0045,.0015)	+.0016
`k4h_v4` + V mean	29.25	.0284 / 93.91 / +.0012	.0151 / 99.62	.0047 / 98.43 / −.0003 (−.0028,.0037)	+.0049
`k4h_v2h` + V mean	22.75	.0408 / 93.00 / +.0028	.0183 / 99.62	.0063 / 97.55 / **+.0043** (.0004,.0111)	+.0193
uniform `k4h_v2h`+vm (bf16, chunk 64)	22.75	.0833 / 89.90 / **+.0132** .0243	.0214 / 99.61	.0177 / 96.27 / **+.0190** (.0116,.0299) .0033 / 98.53 / +.0014	+.0567

Table 9: Methods evaluated and not adopted. "Proxy" = attention-output error on dumped tensors (lower is better; our built layouts in parentheses); all other numbers are measured on the device.

method	evidence	decision
Transforms and quantisers		
KLT/PCA of V and of the key's non-rotary part	V after its mean needs 129–183 of 256 components for 90% of variance; KLT coding at 4.5 bpw has twice int4's proxy error (0.0555 vs 0.0277); dropping 32 of 192 non-rotary key dimensions costs more than all of 4-bit quantisation	rejected
VQ codebooks for V	−30% proxy error vs scalar codes at 3.5 bpw (0.046 vs 0.066), still far above int4 at 4.5 bpw (0.028)	host tier only
Static key channel scaling	after the Hadamard: no proxy gain (int4 0.0280 → 0.0285)	rejected
QJL/TurboQuant-style unbiased K	K is not the bottleneck below 6 bits (proxy `k8_v2h` = `k4h_v2h` = 0.18)	rejected
KIVI granularity (2 bit)	proxy 0.201 at 3 bpw (`k4h_v2h`+vm: 0.104 at 3.5 bpw); KIVI-4 at 5 bpw 0.036 vs `k4h_v4`+vm 0.039 at 4.5; per-channel key groups span eight QSA blocks	rejected
Clipping factor for int4 groups	best $c = 0.95$: −1% (`k4h_v4`+vm 0.0397 → 0.0393); $c \leq 0.9$ worse	rejected
Sparse outlier retention (top 1–2%)	`k4h_v4`: no change; `k4h_v2h`: −9 to −11%, still 2.4× `k4h_v4`, for +0.3–0.5 bpw and a sparse path	rejected
WUSH-style data-adaptive transform	0.0406 / 0.1086 vs Hadamard 0.0397 / 0.1079 (`k4h_v4` / `k4h_v2h`, +vm)	rejected
Best of 16 random rotations (SpinQuant stand-in)	0.0407 / 0.1104 vs Hadamard 0.0397 / 0.1079	rejected
One norm per vector + Lloyd-Max levels (TurboQuant-style)	`k6_v4`: 0.0284 at 5.5 bpw → 0.0320 at 5.06; `k4h_v4`: 0.0397 → 0.0450 at 4.06; at equal bits int5 groups are better (`k5_v4`: 0.0309 at 5.0)	rejected
RoPE dimensions of K at higher precision	64 dims int8 + 192 int4: 0.0380 at 5.5 bpw for K, worse than int5 throughout	rejected
E2M1 (FP4) layout with E4M3 scales per 16, emulated	device: 0.0402 with V mean at 4.5 bpw (+0.0178 above the floor; multi-round ΔNLL +0.0064, significant) vs int4 `k4h_v4` 0.0394 (+0.0162); no FP4 hardware on gfx1201	rejected
10/12-bit layouts	`i8h` already at the floor; beyond it precision matters only on the first layers, where bf16 is exact	not built
Lossless bf16 entropy coding	about 10.3 of 16 bits; `i8h` already at the floor	not built
Sharing across layers		
Cross-layer KV sharing	linear CKA between attention layers 0.52 (V), 0.80 (K)	rejected
Cross-layer selection reuse (CSA2-style)	top-512 overlap with the previous QSA layer only 0.69–0.82; built and measured: reuse on layers 31–47 costs +0.0261 KL above the floor and ΔNLL +0.014 (long) / +0.026 (recall), every other layer +0.0213, all but layer 3 +0.139, for at most 11/13 of the block keys	rejected
Hierarchical indexer (later layers select within layer 3's top-N)	$N = 2048$: +0.0486 KL above the floor, ΔNLL +0.010 / +0.007 / +0.017 (long / recall / multi-round, all significant); $N = 1024$: +0.0869	rejected
Block keys, routing, eviction, demotion		
Block keys in fp8 / PCA to 64 dims	top-512 overlap 0.983 / 0.767 (int8: 0.996, adopted)	rejected
Block keys int6 / int4	top-512 overlap 0.988 / 0.948 (int8: 0.997); int6 saves ∼0.1 KiB/token	low priority
f32 router logits	noise floor 0.0243 → 0.0239; gaps to the floor unchanged	no effect
Token eviction	unbounded error, conflicts with positions and the prefix cache	not adopted
Importance-driven demotion (selection counts)	no better than a position-hash control (Section 4.4, Section 7)	not merged

below the floor because it never reaches the early routing decisions.

Consequences. For this model, KL below about 0.024 against a fixed bf16 computation is meaningful only for a frozen computation path; under varying chunk, batch and parallel shapes, configurations within about +0.002 of the floor are equivalent to bf16. Quantisation should be limited by KL above the floor and by ΔNLL. Whether the floor comes from the MoE routing, from the weight and activation quantisation, or from both, we cannot separate with the models at hand: it would need an unquantised MoE. Evaluating KV quantisation for a quantised MoE against a single bf16 run without measuring this floor risks measuring noise.

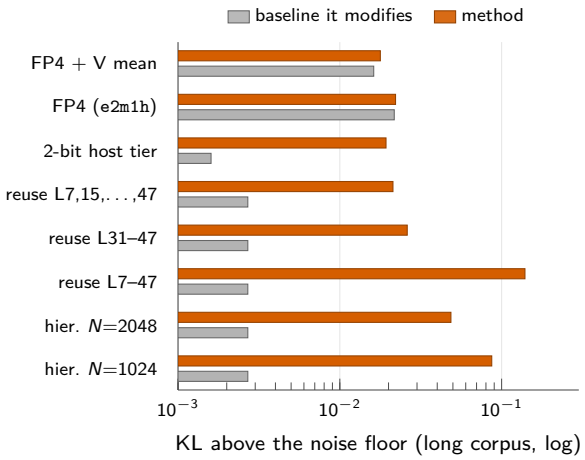


Figure 10: Every rejected device-built method costs more than the baseline it modifies. Baselines: FP4 against int4 `k4h_v4` (with and without V mean), a 2-bit host tier against uniform `i8h`, selection reuse and the hierarchical indexer against `fp8`. Reuse and hierarchical indexing cost 8 to 52 times fp8's own error.

7.8 Decode speed and the drafter under residency

Table 10 measures one sequence on an otherwise idle host: a cold first turn (prompt, then 256 tokens) and a warm second turn of the same conversation (prefix hit, then 512 tokens), with the server's own timings. Without speculation, the recipes decode within the run-to-run spread of fp8 and bf16 (64–90 tokens/s). In a separate pass with speculation at 32K and 128K, each building block alone (`i8h`, `k6_v4`, `k5_v4`, `k4h_v4` with V prediction, int8 block keys, the split, the f16 state) decoded within the spread of fp8 (126–169 against 135–154 tokens/s; an earlier pass, without residency). All measurements in this paper ran at a power cap of 210 W, the minimum of the cards' 210–330 W range; we did not measure at the full limit. Decoding is dominated by the routed experts, and the attention reads only 512 blocks per layer whatever the layout. Window residency costs 3–15% of decode at 32K–250K tokens: the page-in of the selected cold blocks into scratch pages on every QSA layer. Warm time-to-first-token stays below 0.75 s at 250K tokens. The default decodes within the spread of fp8 (77/86 against 78/87 tokens/s at 32K, without speculation), so its extra bits cost no measurable decode speed. The page cache of the pager (Section 3.2) served 62% (32K) and 56% (128K) of the cold entries of fp8 at one sequence without speculation, and decode was 75.2/71.1 against 72.9/69.5 tokens/s without it (+2–3%, within the noise); an 8×64K run gave 52%.

With speculation the default reaches 119–159 tokens/s, and 112–141 with residency. That holds only since a fix found by these measurements. The scheduler built the drafter's serial passes without marking their rows as decode rows, so the pager never ran for the MTP head. The head then read every selected cold block as absent: its acceptance fell from 0.630 to 0.405 for fp8 while the output stayed correct, because the trunk verifies every draft. With the passes' rows staged

Table 10: Residency costs 3–15% of single-sequence decode and nothing in the drafter's acceptance. Decode tokens/s of one sequence (cold turn / warm turn) and warm TTFT, on an idle host; `default` is the near-lossless recipe with layer 47 in bf16.

	residency	32K	128K	250K	TTFT 250K
no speculation					
`bf16`	off	79/90	84/90	72/84	0.71 s
`fp8`	off	78/87	85/89	78/86	0.57 s
`fp8`	on	72/78	72/79	67/82	0.48 s
`default`	off	77/86	78/80	68/76	0.55 s
`default`	on	69/75	69/75	64/74	0.70 s
MTP speculation, 3 drafts					
`default`	off	131/145	159/142	119/131	0.61 s
`default`	on	134/140	136/141	112/131	0.75 s

for the pager, the acceptance with residency equals the acceptance without it, exactly, for fp8 (0.630), the default (0.636) and the compact recipe (0.581). The same bug had made an f16 recurrent state diverge under residency: at f16 the state's anchor replay rounds, so the text depends on how many drafts each step accepts. With identical drafts, residency is again byte-identical for every recipe and knob, including f16 state.

7.9 The MTP drafter shares the last layer's cache

The model's multi-token-prediction head drafts three tokens per step with its own attention layer. It is not a recipe layer: it shares the group of the last attention layer, layer 47, and with it that layer's layout, rotation and calibration maps. The KL measurement runs without speculation and cannot see the head. We therefore measured the drafter's acceptance directly: the share of drafted tokens the trunk accepts, over 16 fixed tasks (2K–25K-token prompts, greedy, up to 512 tokens), with a bootstrap interval over tasks (Table 11).

With layer 47 as `k4h_v4` with V prediction the acceptance is .588, as `k5_v4` rotated with V mean .557, and the compact recipe (layer 47 `k4h_v4` vpred) reaches .581, in single diagnosis runs. Every recipe with at least 6 bits on layer 47 keeps the acceptance of bf16 within its interval. The two search answers that end in `k4h_v2h` vpred on layer 47 cut it from 0.63 to 0.41, and decode with speculation from 178 to 132 tokens/s, although the trunk's KL is the best of all recipes. Layer 47 is the least sensitive layer for the trunk; it is the most sensitive for the drafter. Keeping it in bf16 restores the acceptance (0.636) and also lowers the trunk's KL on all three corpora, for 0.78 KiB/token more; this recipe is our default. A recipe line of its own for the head, and the acceptance as a limit of the search, would make this choice part of the search instead of a fix after it.

7.10 Many long sessions

Table 7 is computed. Table 12 is its measured counterpart: n sessions of one context length, each a prompt plus generated tokens (turn 1) and then a follow-up question (turn 2), with the server's time to first token.

Table 11: 2-bit V on the last layer halves the MTP drafter's gain; bf16 there restores it. Acceptance of drafted tokens (3 per step) over 16 tasks with a 95% bootstrap interval, tokens emitted per step and decode rate with speculation, one sequence.

configuration	layer 47	acceptance	tok/ step	tok/s
`bf16`	bf16	.653 (.620,.687)	2.96	180
`fp8`	fp8	.630 (.597,.667)	2.89	178
`i8h`	`i8h`	.654 (.607,.705)	2.97	180
`max-kl 0.0262`	`k6_v4` rv	.626 (.593,.666)	2.88	172
`max-dkl 0.005`	`k6_v4` rv	.626 (.584,.671)	2.88	173
`max-kl 0.015`	`k4h_v2h` vp	.424 (.389,.467)	2.27	135
`max-kl 0.009`	`k4h_v2h` vp	.412 (.387,.440)	2.23	132
default	bf16	**.636** (.601,.673)	2.90	174

Without residency, fp8 holds 16 sessions of 128K tokens (follow-ups in 11/19 s, the conversations still in the prefix cache), but 32 sessions evict the prefix cache and every follow-up is recomputed: 1 843/3 330 s. An early residency build gave no capacity at all: its sweep ran only after decode-only steps with a small block budget and demoted about 2% of the middles (10.7K of about 500K blocks). After the sweep ran after every step under a byte budget, with sparse mixed steps promoting only prefill rows, all middles were demoted (494 679 blocks at 16 sessions), and follow-ups returned in 20/31 s. With one store per group at exact slot sizes, the compact recipe held 32 sessions of 128K tokens in a 48 GiB store, every follow-up hit the prefix cache, and the follow-ups took 58/96 s. fp8 at 32×128K needs about 50 GiB of store (1.01 M blocks of 52 KiB) and did not fit in 48 GiB; the default recipe at 16×128K did not fit before the stores had exact slot sizes (their slots were bf16-sized). Once the blocks the prefix cache holds were demoted as well, the default recipe at 8×64K skipped none (the count had been in the millions), kept the KV pool at 25–31% utilisation, and returned follow-ups in 6.2/8.0 s and 10.8/12.5 s (two turns). So the host store, not VRAM, bounds the number of sessions here. We did not measure VRAM per sequence directly, and report single configurations, not a sweep of session counts per card.

7.11 Decode while prefilling

Table 13 measures eight sessions decoding with the default recipe while one or four 64K-token prompts prefill (Section 3.3). Without interleaving, a decoding session falls from 73 to 12 tokens/s beside one prompt and to 5 beside four, and the prompts prefill fastest (3 534 tokens/s aggregate). With `auto` the decoders keep 23 (15), with `auto:1` 42 (33) and with `auto:2` 50 (36) tokens/s, and the prefill throughput falls to 3 181, 1 758 and 1 166 tokens/s (one prompt), with the prompt's time to first token rising from 18 s to 20, 37 and 56 s. Shrinking the chunk while decoders are busy (1024 tokens) did worse on both axes. The setting is therefore a trade, not a gain: `auto:1` doubles a long prompt's wait to give a decoding session three times its rate, and an operator who serves mostly long-prompt batch work may prefer 0. Short prompts fit one chunk and are not held: with the default recipe at concurrency 1/2/4/8 the aggregate rate was 155/230/319/392 tokens/s under `auto` against 150/194/249/341 with interleaving off, a difference we do not consider reliable (two server starts).

The decode headroom removes the remainder steps. In separate runs with one prompt it raised the prefill throughput by 22% both without interleaving (3 664 to 4 455 tokens/s, TTFT 17.3 to 14.2 s, session decode 12.3 to 9.5 tokens/s) and with `auto:1` (1 823 to 2 217, TTFT 35.5 to 29.0 s, decode 40.9 to 39.1). The cost is the decode rate when interleaving is off, since the decoders lose the cheap remainder steps. A single 64K prefill alone runs at about 5.1K tokens/s, beside eight decoders at 3.5K. The scheduler simulation, the real scheduler driven by a step-cost model fitted to a lone prefill and to decode alone, predicted a time to first token of 17/20/33/51 s for 0/`auto`/`auto:1`/`auto:2` against 17.3/20/35.5/56 s on the device (headroom 0, one prompt); the schedule is the real one, the times are estimates.

Table 12: Residency turns a 32×128K workload from tens of minutes of recomputation into follow-ups of about a minute. Turn-2 time to first token (p50/p95) for n sessions of one context length, Qwen3.8-Flash-Next at tensor parallelism 2, power cap 210 W, MTP with 3 drafts. Aggregate decode rate in tokens/s where measured. Residency rows use a 48 GiB store. The note says whether the follow-ups hit the prefix cache (*cached*, *all hit*) or had to be recomputed (*evicted*), that every middle was demoted, that the store could not hold the demoted blocks (*no fit*: fp8 at 32×128K needs about 50 GiB), or that the sweep skipped no block (KV utilisation 25–31 %).

configuration	n×ctx	turn-2 TTFT (s)	tok/s	note
`fp8`, no residency	16×128K	11/19	239	cached
`fp8`, no residency	32×128K	1 843/3 330	–	evicted
`fp8`, residency	16×128K	20/31	197	all demoted
compact, residency	32×128K	**58/96**	262	all hit
`fp8`, residency	32×128K	–	–	no fit
default, residency	8×64K	6.2/8.0	–	skipped 0

Table 13: Interleaving trades prompt throughput for decode rate: `auto:1` lifts a decoding session from 12 to 42 tokens/s during a 64K prefill and doubles the prompt's wait. Eight sessions decoding (default recipe, MTP with 3 drafts) while P prompts of 64K tokens prefill; decode rate per session alone → during; prefill rate summed over the prompts; median time to first token of the prompts. Separate server starts, so "alone" varies by a few tokens/s. These runs predate the decode headroom (equivalent to `-decode-headroom 0`); its effect is in the text.

`-prefill-interleave`	P	decode (tok/s)	prefill (tok/s)	TTFT (s)
0	1	73 → 12	3 534	18
`auto` (share 0.25)	1	83 → 23	3 181	20
`auto:1` (served default)	1	78 → 42	1 758	37
`auto:2`	1	79 → 50	1 166	56
0, busy chunk 1024	1	73 → 8	2 298	28
0	4	72 → 5	2 934	64
`auto`	4	72 → 15	2 545	74
`auto:1`	4	77 → 33	1 395	137
`auto:2`	4	79 → 36	930	204

8 Negative results

Table 9 lists the methods we evaluated and did not adopt; Fig. 10 shows the measured cost of those built on the device. Three lessons stand out. First, the fixed Hadamard rotation is not beaten by any transform we tried, data-adaptive or random. Second, the selections of neighbouring QSA layers differ too much for a model not trained to share them: both selection reuse and a hierarchical indexer cost significant likelihood on every corpus, for at most 0.2 KiB/token saved. Third, an importance signal for demotion was no better than a hash of the position.

9 Discussion and limitations

What the frontier says. Three mechanisms produce the frontier of Fig. 6: depth-dependent precision (early layers need 8 bits or more), exact zero-bit transforms (rotation, V mean, V prediction), and narrowed block keys where the recipe is already at the floor. None of them needs training. The search's second pass matters as much as the layouts: without it the tool cannot see recipes that narrow from a uniform 8-bit layout, and the fp8-equivalent recipe had to be found by hand.

Limitations. All measurements use one model on one two-card platform, at a power cap of 210 W (the minimum of 210–330 W); speed at the full limit was not measured. All KL is relative to the same weight- and activation-quantised model with a bf16 KV cache. The corpora have four documents each; t-intervals over documents are three to six times wider than the position-block bootstrap, although the sign of every reported difference agrees in all four documents. Single-sequence speed was measured for cold and warm turns at three context lengths; the many-session runs are single configurations, so sessions per card are not established systematically, and VRAM per sequence (Table 7) is computed. The V prediction obtains $P\hat{K}$ from a second attention pass, so its decode cost is not minimal. Tiered precision was measured only with an `i8h` resident layout and its speed was not measured; it is not in the upstream submissions (Section 11), and we have not re-run tiered layers with int8 block keys and the MTP drafter since the drafter's pager fix (Section C). A prefill row promotes its sequence's whole cold middle although a sparse row reads only its selection, which is why a returning conversation pays for a copy-back; the drafter's history pass right after a prompt chunk pages only the entries that fit the scratch rows (it affects drafts, never a token). The scheduling trade (Section 7.11) is global to the server, follows measured times and is not reproducible run to run under `auto`; its simulation is fitted to the device, not measured per step shape. Residency costs 3–15% of single-sequence decode, and the page-in cannot be overlapped with the preceding linear-attention layers.

Future work. A fused attention kernel for the V prediction; a systematic capacity table of sessions per card at 32K, 64K and 128K tokens; a 16-document corpus; divergence timing of long greedy generations against a bf16 noise baseline; teacher-forced per-layer cost as the search's cost model; serving prefill rows from the store instead of promoting; speculative prefetching of pages; and a recipe line and search limit for the MTP head.

10 Conclusion

Sparse attention lets a KV memory system keep only a small, predictable part of each sequence on the device and page the rest on demand without changing any output, so that context is bounded by host memory and per-sequence VRAM grows only by the block keys. On top of that, per-layer and per-tier quantisation with free rotation, V-mean and V-prediction transforms reduces the per-token footprint by 22% at fp8's quality and by 44% within the noise floor +0.01, at no measurable likelihood cost on the long corpus. The recipes come from AutoQuant, a search that trusts only measurements on the device: its summed per-layer costs propose, its runs decide, and a second pass around the best measured recipe finds what the costs from the unquantised base cannot see. Residency costs 3–15% of single-sequence decode and, with all middles demoted, makes the host store the limit on sessions: 32 sessions of 128K tokens with the compact recipe returned follow-ups in 58/96 s against 1 843/3 330 s without residency. While long prompts prefill, an interleave policy trades prompt throughput for the decoders' rate. For a quantised MoE model, every such claim has to be read against the model's own numerical noise floor, which paired measurement makes visible.

11 Availability

The engine is radiance; the work of this paper is in `github.com/Code000-211/RadianceMemory`. Four parts were submitted upstream to the engine's repository (`codeberg.org/StillDeadcode/radiance`): #410, `-prefill-interleave auto[:SHARE]` (Section 3.3); #411, `-decode-headroom`; #412, window residency; and #416, the KV layouts, the transforms, `rad-autoquant` and the four preset recipes (Section D). Tiered precision is not part of these submissions.

References

[1] S. Yang, J. Kautz, A. Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. arXiv:2412.06464, 2024.

[2] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. arXiv:2309.06180, 2023.

[3] Z. Liu et al. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. arXiv:2402.02750, 2024.

[4] C. Hooper et al. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. arXiv:2401.18079, 2024.

[5] S. Ashkboos et al. QuaRot: Outlier-free 4-bit inference in rotated LLMs. arXiv:2404.00456, 2024.

[6] Z. Liu et al. SpinQuant: LLM quantization with learned rotations. arXiv:2405.16406, 2024.

[7] J. Chen, V. Egiazarian, E. Kurtić, T. Hoefler, D. Alistarh. WUSH-KV: KV cache quantization with data-adaptive transforms. arXiv:2609.38121, 2026.

[8] KVarN: variance normalisation for low-bit value caches. arXiv:2606.03458, 2026.

[9] A. Shutova et al. Cache me if you must: Adaptive key-value quantization for large language models. arXiv:2501.19392, 2025.

[10] A. Zandieh et al. TurboQuant: Online vector quantization with near-optimal distortion rate. arXiv:2504.19874, 2025.

[11] NVIDIA. Model Optimizer: AutoQuant KV, automatic mixed-precision KV cache quantization. Documentation, 2026.

[12] G. Xiao et al. Efficient streaming language models with attention sinks. arXiv:2309.17453, 2023.

[13] Z. Zhang et al. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. arXiv:2306.14048, 2023.

[14] Y. Li et al. SnapKV: LLM knows what you are looking for before generation. arXiv:2404.14469, 2024.

[15] DistillCache: KL-guided adaptive KV-cache eviction for memory-efficient LLM inference. arXiv:2608.08878, 2026.

[16] DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression. arXiv:2609.19969, 2026.

A QSA implementation details

Tail designs that fail. Three simpler designs of the raw-key tail produce fluent but wrong text: a buffer shared across layers (a layer pools the previous layer's keys), a plane indexed by batch row (a request inherits a retired request's keys) and four rows indexed by position modulo four (a rejected speculative draft is overwritten by a future token's key). The tail used is a linear-state group of $4 + n_{\text{spec}}$ rows per layer and sequence, written by position and checkpointed with the recurrent state.

Prefill. At prefill attention is issued one row per query, each with its own selection; the partial block is appended by position, not by table content.

B Window residency internals

Figure 11 shows the states a block of the attention group passes through.

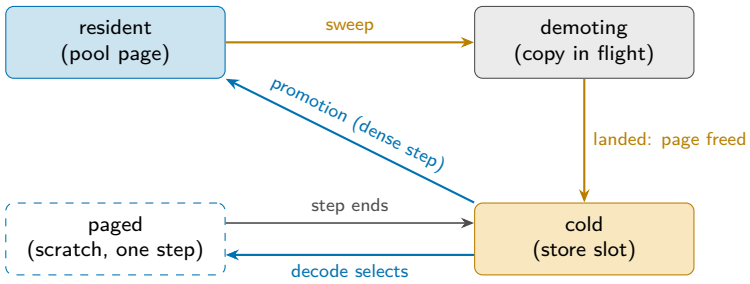


Figure 11: States of an attention block under window residency. An unchanged promoted block returns to cold by re-pointing to its origin slot, without a copy.

Scratch pages. Each decode row owns $513\,(1 + n_{\text{spec}})$ scratch pages, reserved when the pool is carved (about 208 MiB per card at eight decode rows). A page keeps the block it last received, tagged with its store slot per layer; a later selection naming that slot is rewritten to the page with no copy. A planning kernel finds the hits and a fill kernel tags a page only after its copy. The tags are zeroed whenever a slot is freed, the store changes or the pool is carved, so a tag never names a reused slot; `RADIANCE_QSA_PAGE_CACHE=0` turns the cache off for comparison.

Stream-ordering rules of the sweep. The sweep demotes within its byte budget after each committed step, across all sequences, on a transfer stream. Four rules make this safe without any host wait: (1) a block's pool page is freed only after its copy has landed; the copy waited on the compute stream, so every step that could read the page in place was queued before it; (2) a freed page's new owner writes it only in a later step on the same in-order compute stream; (3) a promotion waits on the compute stream before writing a page the pool just handed out, which a step issued earlier may still read under its previous owner; (4) one sweep in flight, because landing is a poll of the newest transfer and a sweep every step would keep it pending forever. A step that will page a block in additionally waits on its demotion's completion event on the stream (no host synchronisation).

Dense steps and preemption. Attention reads the table densely for a whole step only while no entry lies beyond the window plus one block, and a prefill row, which the per-decode-row scratch cannot serve, always does. Prefill rows are therefore promoted before their step, and decode rows only when the step may be dense: fresh pages, copy back, compute waits on the copy. In a sparse mixed step the decode rows keep paging. Promotion is all or nothing per sequence. If the pool cannot hold a sequence's cold blocks even after the prefix cache has given up room, *that* sequence is preempted and recomputed. Preempting the back of the running set instead would usually hit the prompt that made the step mixed, which would be readmitted into the same failing step while the cold sequence never decodes, a livelock now covered by a test.

Prefix cache. Store slots are reference counted, so a cold entry can sit in several sequence tables and prefix-cache entries; a slot is freed when its last holder releases it. A conversation returning for its next turn hits the prefix cache including its cold middle, adopts the slots and is promoted by copy rather than recomputed: in an early measurement on an older engine base, a warm 8 887-token prefill took 92.7 ms instead of 1 601 ms (70.1 ms without residency). Blocks that the cache shares with a live sequence are demoted and the cache entry follows them into the store, and an idle conversation held by the cache alone leaves the card whole; before this, such blocks were skipped (in the millions in the 8×64K run). A promoted block remembers its origin slot; since prefix blocks are immutable, demoting it again re-points the entry to that slot without moving bytes, so the store does not accumulate a copy of the same prefix per turn.

Memory accounting and tensor parallelism. Scratch pages are fixed costs charged before the VRAM split. There is one store per residency group; its slot holds one block of that group, every rank's shard at its own offset, at the group's exact size. At 200K tokens a sequence keeps 640 attention blocks resident instead of 50 000; its block keys (159 MiB at bf16, 84 MiB in int8) and its recurrent state (54 / 27 MiB in f32 / f16) stay beside them; sink and tail take about 16 MiB. A slot-pitch error under tensor parallelism, where the pager walked the store at one rank's share instead of the whole slot, was found and fixed during the quantisation work.

Relation to KV streaming. radiance also ships KV streaming, which keeps the authoritative copy in host memory and a VRAM ring of recently read blocks (CLOCK eviction), active only above a 32 768-token ring. Window residency keeps the authoritative copy in VRAM and demotes the predictable middle, works from about 2.6K tokens of context, and keeps the prefix cache's host tier; the two are alternatives.

Cost. The long-context cost is in Section 7.8: 3–15% of single-sequence decode. An early measurement on an older engine base at short contexts (two passes of a warm-cache benchmark) showed warm prefill paying for copying back the conversation's cold middle (39.2 → 42.4 ms at 3 013 tokens, 70.1 → 92.7 ms at 8 887); we do not use its decode figures, which predate the pager's current form. The page-in cannot be overlapped with the preceding GDN layers: a layer's selection is computed by its own indexer from its input right before its attention. Overlap would require speculative prefetching, for example of the previous step's selection.

The drafter under residency. The scheduler built the MTP drafter's serial passes without marking their rows as decode rows, so the pager never ran for the head and it read every selected cold block as absent. Its acceptance fell from 0.630 to 0.405 for fp8; the text stayed correct because the trunk verifies every draft, so the loss showed only as speed. The same bug explains why an f16 recurrent state diverged under residency: the state's anchor replay rounds at f16, so the generated text depends on how many drafts each step accepts, and residency had changed the acceptance. With the rows staged for the pager the acceptance is equal (fp8 0.630, default 0.636, compact 0.581) and the text is byte-identical for every recipe and knob tested.

C Verification

Every new operator (Hadamard rotation, packed stores and attention, transcode, the expanding pager, int8 block keys, the f16-state recurrence, the block-key split and V prediction) has an exact libref reference. GPU kernels are compiled for a fiber-based host emulator of the HIP runtime and compared with the reference byte for byte, and on the device by `r4d_selftest`; attention cases with a decisive key are byte-equal, others agree within rounding because the kernels accumulate in f16 and merge partials. Tests were checked to fail on injected faults. The decode rows of `k5_v4` and `vpred` first failed only the selftest's scratch-band check, which lacked the new row widths (fixed in the selftest). The hierarchical-indexer kernels passed their selftests (pool mask 2/2, selection 18/18).

End to end, greedy output (256 tokens on a 12 452-token prompt) with residency on is compared byte for byte with residency off, over the whole reply (an earlier script compared only a reasoning model's empty `content` field and was fixed), for fp8, int8 block keys and mixed recipes, with and without the MTP drafter. After the drafter's pager fix the comparison, with the drafter on, also passes for fp8 with an f16 state, the default and the compact recipe with all its knobs (an earlier compact run with the drafter on diverged after about 30 tokens). Tiered recipes equal their emulation byte for byte (bf16 block keys). The work found and fixed three latent bugs: the tensor-parallel store pitch, a pinned index buffer shared by all batches of a sweep and read late by the transfer stream (which made residency nondeterministic), and a missing barrier in the pager before rewriting a selection entry. Validating the search answers found a fourth, in the block-key split. The engine binds a KV group to the first unclaimed op that states a block size, and the plugin declares all attention groups before any op. With five or more layouts in a recipe, one group bound a later layer's `qsa_block_key`, which under the split states 8-token blocks, and that layout's attention refused the shape. Unsplit, every block size is 4 and the mis-binding was invisible. The block-key group now binds to its own first op immediately, and only that op states the size. With the split, the `-max-kl 0.015` recipe is byte-identical to the same recipe without it, and so is fp8. One combination is unverified: tiered layers together with int8 block keys and the MTP drafter diverged between residency and its emulation (they matched without speculation), with the drafter's acceptance falling from about 0.68 to 0.10 once blocks actually moved. The untiered form of that fall was the drafter's missing pager, fixed afterwards; we did not re-run the tiered combination.

Upstream sync: after merging 146 upstream commits into the engine tree, the device selftests ran 43 rows with no failure, the per-position KL rows were byte-identical to the rows before the merge for fp8, `i8h`, `k5_v4` rotated, `k4h_v4` with V prediction, and int8 block keys with the split, and the residency comparison passed for fp8, the default and the compact recipe. For the submission of the layouts (#416), 46 selftest rows ran with no failure and the KL rows were byte-identical; for the merged tree, `qsa_page_u8` passed 23 of 23 rows, `qsa_select` 18 of 18, `part/join` 4 of 4 and `attn_decode_scratch` 40 of 40.

The CPU test suite passes except for one test that exceeds its 120 s timeout under parallel load and passes alone (202 cases, 0 failures); the out-of-tree plugin test passes (202 fixture ops, 0 failed) but takes 3 698 s against its 3 600 s limit on the build machine.

D Recipes and serving

`-kv-recipe FILE` lists one line per attention layer: `layer N LAYOUT [rotated] [demote LAYOUT] [vmean] [vpred]`, plus file lines `vmean PATH` and `vpred PATH` for the calibration data. A uniform recipe is byte for byte the corresponding global cache type, mixed layouts become one KV group per distinct layout, the MTP head follows the last attention layer, and window residency sizes the store per layout. Refusals name the layer and line. The compact recipe of Table 1, for example, is served with

```
-kv-cache-dtype fp8 -kv-recipe qwen3.8-flash-next-compact.kvrecipe -qsa-bkey-dtype i8
-state-cache-dtype f16 -qsa-bkey-split on
```

and the fp8-equivalent recipe without the state option. Four presets ship with the engine as answers of the search for this model: *near-lossless* (the default; 9.56 KiB/token, KL 0.0065, MTP acceptance 0.636), *fp8-equivalent* (5.28, 0.0257, 0.626), *floor-0.005* (4.79, 0.0281, 0.626) and *compact* (3.76, 0.0335, 0.581), with the values of Table 1. The near-lossless preset serves without int8 block keys; the compact preset needs the f16 state. A preset is calibrated for this model only, and the `vpred` file (22 MiB) is made once by the user with a calibration run. The served scheduler defaults are `-prefill-interleave auto:1` and `-decode-headroom auto` (Section 3.3), and residency is off unless `-kv-residency-mib` is given. The search (Section 5) is invoked as `rad-autoquant -max-kl K` or `-max-dkl D` (with `-max-dnll`, `-min-top1`, `-bits` and `-strict` as further limits) and writes the recipe together with every measurement it made as comments.

E Additional measurements

E.1 Every configuration with the floor removed

Each configuration was run again at prefill chunk 64; "above the floor" is its KL minus the bf16 chunk-64 run's, paired per position, 95% block bootstrap.

configuration	bpw	KL alone (chunk 2048)	KL with the floor (chunk 64)	above the floor (95%)	top-1 with the floor (%)
bf16 (the floor itself)	16	0.0000	0.0243	+0.0000 (+0.0000, +0.0000)	94.74
`i8h`	8.06	0.0236	0.0259	+0.0016 (+0.0005, +0.0028)	94.44
`i8h` + vmean	8.06	0.0243	0.0255	+0.0012 (+0.0002, +0.0022)	94.38
`fp8`	8.0	0.0262	0.0270	+0.0027 (+0.0015, +0.0038)	94.37
`fp8` rotated	8.0	0.0270	0.0266	+0.0023 (+0.0014, +0.0035)	94.13
`fp8_v4`	6.25	0.0368	0.0367	+0.0124 (+0.0107, +0.0141)	93.38
`fp8_v4` + vmean	6.25	0.0342	0.0340	+0.0097 (+0.0078, +0.0116)	93.72
`fp8_v4` rotated	6.25	0.0364	0.0356	+0.0113 (+0.0098, +0.0132)	93.24
`fp8_v4` rotated + vmean	6.25	0.0320	0.0318	+0.0075 (+0.0060, +0.0091)	93.96
`k6_v4`	5.5	0.0344	0.0353	+0.0109 (+0.0096, +0.0124)	93.56
`k6_v4` + vmean	5.5	0.0321	0.0346	+0.0103 (+0.0087, +0.0117)	93.57
`k6_v4` rotated	5.5	0.0352	0.0354	+0.0111 (+0.0095, +0.0129)	93.76
`k6_v4` rotated + vmean	5.5	0.0309	0.0307	+0.0064 (+0.0050, +0.0078)	94.18
`k4h_v4`	4.5	0.0437	0.0460	+0.0217 (+0.0193, +0.0243)	92.54
`k4h_v4` + vmean	4.5	0.0394	0.0405	+0.0162 (+0.0136, +0.0185)	93.32
`k4h_v2h`	3.5	0.1372	0.1356	+0.1113 (+0.0987, +0.1222)	87.55
`k4h_v2h` + vmean	3.5	0.0833	0.0810	+0.0567 (+0.0503, +0.0627)	90.16
`fp8` + int8 block keys	8.0	0.0256	0.0278	+0.0035 (+0.0020, +0.0044)	94.35
`fp8` + int8 block keys + f16 state	8.0	n/a	0.0280	+0.0037 (+0.0026, +0.0050)	94.06
`i8h` + int8 block keys	8.06	0.0243	0.0250	+0.0007 (−0.0001, +0.0016)	94.38
prev. default (`k6_v4` rot+vm, i8 keys, f16 st.)	5.5	n/a	0.0318	+0.0075 (+0.0060, +0.0092)	93.89
A: `i8h` L3–23, `k6_v4` rot+vm L27–47	6.78	0.0255	0.0273	+0.0030 (+0.0019, +0.0043)	94.15
A + int8 block keys (fp8-equivalent)	6.78	0.0260	0.0263	+0.0020 (+0.0011, +0.0031)	94.35
G: `i8h` L3–19, `k6_v4` rot+vm L23–47	6.57	0.0263	0.0280	+0.0036 (+0.0026, +0.0048)	94.19
bf16 L3–23, `i8h` L27–47	12.03	0.0061	0.0243	+0.0000 (−0.0004, +0.0005)	94.64
first-pass search `-max-kl 0.015`	10.88	0.0075	0.0254	+0.0011 (+0.0006, +0.0017)	94.40
first-pass search `-max-kl 0.009`	11.54	0.0051	0.0256	+0.0013 (+0.0010, +0.0018)	94.26
tiered `i8h` / `k4h_v4`+vm (residency)	8.06	0.0284	0.0292	+0.0049 (+0.0036, +0.0062)	93.94
tiered `i8h` / `k4h_v2h`+vm (residency)	8.06	0.0408	0.0436	+0.0193 (+0.0162, +0.0226)	92.94

E.2 Global knobs and further corpora

On `fp8` / `i8h` KV, int8 block keys give 0.0256 / 0.0243 (bootstrap intervals of the change include 0) and the f16 GDN state 0.0269 / 0.0253; at prefill chunk 64, fp8 goes from 0.0270 to 0.0274 with the f16 state. The new layouts on all three corpora (KL / top-1 / ΔNLL, * = significant):

configuration	bpw	long	recall	multi-round	above floor (long)
`e2m1h` (FP4, emulated)	4.5	.0439 / 92.80 / +.0022	.0123	.0081 / +.0031	+.0221
`e2m1h` + vmean	4.5	.0402 / 93.14 / +.0026	.0123	.0071 / +.0064*	+.0178
`k5_v4` rot+vm	5.0	.0328 / 93.74 / −.0006	.0105	.0055 / +.0016	+.0084
`k5_v4`	5.0	.0374 / 93.26 / +.0043	.0149	.0061 / +.0086*	+.0147
`k4h_v4` vpred	4.5	.0373 / 93.24 / +.0054	.0154	.0062 / +.0048*	+.0126
`k4h_v2h` vpred	3.5	.0555 / 91.87 / +.0083*	.0295 / +.0240*	.0128 / +.0139*	+.0296

E.3 Selection reuse and hierarchical indexer, all corpora

fp8 KV; KL / top-1 / ΔNLL (bold = significant).

variant	block keys saved	long	recall	multi-round	above floor
none (`fp8`)	0	.0262 / 94.35 / +.0010	(`i8h` .0103)	(`i8h` .0032)	+.0027
reuse L31–47 (5 layers)	5/13	.0490 / 93.11 / **+.0140**	.0424 / 98.94 / **+.0260**	.0231 / 98.24 / +.0187	+.0261
reuse L7, 15, . . . , 47 (6)	6/13	.0467 / 92.79 / **+.0090**	.0213 / 99.54 / +.0033	.0045 / 98.63 / +.0013	+.0213
reuse L7–47 (11)	11/13	.1673 / 87.09 / **+.0662**	.0507 / 99.06 / **+.0169**	.1348 / 93.33 / **+.1427**	+.1394
hierarchical, pool $N = 2048$	–	.0701 / 91.28 / **+.0099**	.0192 / 99.46 / **+.0074**	.0161 / 97.16 / **+.0172**	+.0486
hierarchical, pool $N = 1024$	–	.1112 / 89.09 / **+.0282**	.0319 / 99.24 / **+.0189**	.0839 / 94.80 / **+.0914**	+.0869

In the proxy, a layer's top-512 lies 79–93% inside layer 3's top-1024 and 93–99% inside its top-1536; overlap with layer 3's own top-512 is 0.51–0.69.

E.4 f32 router

Table 14: Noise floor and gaps with f32 router logits and top-k (new bf16 reference with the f32 router) against the bf16 router. Δ is the gap to the respective floor (±SE).

	f32 router				bf16 router			
config	mean KL	p99	top-1	Δ	mean KL	p99	top-1	Δ
floor (bf16, chunk 64)	.0239	.246	.9473	+.0000	.0243	.239	.9474	+.0000
`fp8`	.0255	.258	.9437	+.0016 ±.0005	.0262	.267	.9435	+.0019 ±.0005
uniform `i8h`	.0231	.228	.9473	−.0008 ±.0005	.0236	.236	.9477	−.0008 ±.0005
`k6_v4` rot+vm	.0307	.307	.9404	+.0068 ±.0006	.0309	.321	.9409	+.0066 ±.0007
bf16, f16 state	.0239	.248	.9474	−.0000 ±.0005	.0245	.242	.9447	+.0002 ±.0005

E.5 Proxy studies

Attention-output error against bf16 K/V (rank 0, 12 layers, prose prefill, calibration on a code prefill).

- **`k4h_v2h` threshold factor** α (boundaries at 0, $\pm\alpha$ rms): $\alpha = 0.6$ / 0.8 / 0.9 / **1.0 (built)** / 1.1 / 1.2 / 1.4 gives 0.1320 / 0.1151 / 0.1102 / **0.1079** / 0.1086 / 0.1113 / 0.1219; a least-squares scale is worse at every α (0.1120 at $\alpha = 1$).
- **Block-diagonal Hadamard** keeping the 64 RoPE dimensions apart: $H_{64} \oplus H_{64} \oplus H_{128}$ 0.0397 / 0.1079, $4 \times H_{64}$ 0.0401 / 0.1077, whole-head H_{256} (built) 0.0397 / 0.1079, no rotation 0.0469 / 0.1584 (`k4h_v4`+vm / `k4h_v2h`+vm).
- **Clipping factor** c (scale $= c \cdot \text{absmax}/q_{\max}$): `k4h_v4`+vm 0.0397 / 0.0393 / 0.0434 / 0.0527 / 0.0659 at $c = 1$ / 0.95 / 0.9 / 0.85 / 0.8; `k6_v4` rot+vm and int8 only get worse; `k4h_v2h`'s K 0.1079 → 0.1074.
- **Outlier retention** (top 1% / 2% of values per token row exact): `k4h_v4`+vm 0.0397 → 0.0397 / 0.0395; `k4h_v2h`+vm 0.1079 → 0.0984 / 0.0958.
- **Finer grid**: `k5_v4` 0.0309 between `k6_v4` (0.0284) and `k4h_v4` (0.0397); any 3-bit V is poor (`k6_v3` 0.0658, `k5_v3` 0.0668, `k4h_v3` 0.0715).
- **V predicted from K**: `k4h_v4` 0.0397 → 0.0355 (−11%), `k4h_v2h` 0.1079 → 0.0798 (−26%); on the device −22% / −48% above the floor.
- **Per-vector norm**: `k6_v4` 0.0284 → 0.0320 (Lloyd-Max) / 0.0335 (uniform) at 5.06 bpw; `k4h_v4` 0.0397 → 0.0450 / 0.0482 at 4.06. RoPE part of K in int8 / int6: 0.0380 / 0.0381 at K 5.5 / 5.0 bpw.